

Deep Reinforcement Learning applied to Forex Market: Deep Deterministic Policy Gradient Trading System

PIC2 - Master in Computer Science and Engineering
Instituto Superior Técnico, Universidade de Lisboa

Vicente Aser Marques Rebelo Castillo Lorenzo — 92569*
vicente.aser.lorenzo@tecnico.ulisboa.pt

Advisor: Rui Fuentecilla Maia Ferreira Neves

Abstract

Most published work on machine learning for currency trading reports what a strategy earned rather than what it would have kept. This dissertation asks whether a reinforcement learning agent can trade the seven major currency pairs profitably once it is charged what a real account charges, and whether the answer holds when one design is applied to every pair without adjustment. Answering it required building the environment first: a trading framework in which a single strategy implementation runs offline, inside the broker platform's own backtester and on a live account, with an execution engine that steps on individual quotes rather than bar closes, reproduces the commercial platform it was built against byte for byte, and charges the spread recorded at the moment of each trade together with the commission of a swap-free raw-spread retail account, drawing on a database of 1.66 billion quotes. Within that environment one Deep Deterministic Policy Gradient (DDPG) agent was trained per pair, observing thirty-eight dimensionless market features and emitting a single continuous action that sets both the direction and the size of a position, which the discrete action spaces used in almost all prior currency work cannot express. Over eleven years all seven agents are profitable, returning between 15.7% and 226.6% net of costs from five different starting balances, and since five of the seven pairs lost value over that period most of this return was earned against falling markets. Six pairs show positive alpha, and on three of them the market exposure, measured as beta against the pair's own movement, is close to zero, while combining the seven into an equal-weight portfolio cuts the maximum drawdown from 34.5% to 15.9%. Two findings generalise: the agents win barely more often than a coin, so the edge lies in sizing rather than prediction, and in every pair the highest-returning candidate failed a permutation test, meaning that selecting on return alone yields market exposure dressed as skill.

Keywords — Forex Trading, Deep Reinforcement Learning, Deep Deterministic Policy Gradient, Transaction Costs, Portfolio Construction

*I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa (<https://nape.tecnico.ulisboa.pt/en/apoio-ao-estudante/documentos-importantes/regulamentos-da-universidade-de-lisboa/>).

Acknowledgements

Contents

1	Introduction	5
1.1	Motivation	5
1.2	Problem Statement	5
1.3	Objectives	6
1.4	Contributions	6
1.5	Structure	7
2	Background	9
2.1	Financial Markets and Trading	9
2.1.1	Forex Market: Characteristics and Significance	9
2.1.2	Forex Trading: Fundamental Concepts and Operation	9
2.1.3	Forex Costs: Spreads, Commissions and Overnight Financing	10
2.1.4	Financial Analysis: Main Types and Formulas	11
2.1.5	Performance Measurement: Returns, Risk and Ratios	14
2.1.6	Algorithmic and Quantitative Trading	15
2.2	Neural Networks	15
2.2.1	Fundamentals and History	15
2.2.2	Key Concepts and Feed-Forward	16
2.2.3	Learning using Backpropagation	17
2.2.4	Bias-Variance Trade-off and Model Generalisation	17
2.2.5	Architectures for Sequential Data	18
2.2.6	Universal Approximation Theorem	18
2.3	Reinforcement Learning	19
2.3.1	Fundamentals and History	19
2.3.2	Key Concepts and Agent-Environment Interaction	19
2.3.3	Value Functions and Q-Functions	20
2.3.4	Methodological Variants and Learning Methods	21
2.4	Deep Reinforcement Learning	22
2.4.1	Fundamentals and History	22
2.4.2	Deep Value-Based RL Algorithms: Deep Q-Networks	22
2.4.3	Deep Policy-Based RL Algorithms: Deep Policy Gradients	23
3	Related Work	25
3.1	Survey Research: Identifying the State-of-the-Art	25
3.2	Article Research: Summarising Different Implementations	25
3.2.1	Forex Market Implementations	25
3.2.2	Stock Market Implementations	28
3.2.3	Other Market Implementations	28
3.2.4	Comparative Findings	28
3.3	Research Gap: Positioning This Work	29
4	Implementation	31
4.1	Trading Framework: The Environment the Agent Trades In	31
4.1.1	System Architecture: Three Execution Contexts	31
4.1.2	Market Data: Collection, Storage and Resolution	31
4.1.3	Cost Model: Spread, Commission and Account Type	32
4.2	Agent Design: Observation, Action and Reward	34
4.2.1	Observation Space: Indicators, Encoding and Normalisation	34

4.2.2	Action Space: Position Sizing and Risk	36
4.2.3	Reward Function: Log Return, Scaling and Clipping	36
4.2.4	Learning Algorithm: DDPG and its Adaptations	36
4.3	Experimental Procedure: Training, Selection and Deployment	38
4.3.1	Training Procedure: Seeds, Episodes and Data Splits	38
4.3.2	Model Selection: Gates and Composite Ranking	39
4.3.3	Deployment: Decision Cadence and Rebalance Threshold	40
4.3.4	Worked Example: One Position From Decision to Settlement	41
5	Experimental Results	43
5.1	Evaluation Protocol	43
5.2	Benchmarks and Performance Metrics	43
5.3	Results Across the Seven Majors	44
5.3.1	AUD/USD	45
5.3.2	EUR/USD	46
5.3.3	GBP/USD	46
5.3.4	NZD/USD	46
5.3.5	USD/CAD	46
5.3.6	USD/CHF	47
5.3.7	USD/JPY	48
5.4	Trading Behaviour: How the Agents Actually Trade	48
5.4.1	Use of the Continuous Action	49
5.4.2	Direction, Size and Outcome Together	50
5.4.3	Trend Identification and Reversal	50
5.4.4	Holding Periods and Two-Sided Exposure	51
5.4.5	Where the Edge Comes From	52
5.4.6	What the Trading Costs	52
5.5	Path Robustness	53
5.6	Alpha and Beta Decomposition	54
5.7	Statistical Screening and the Rejected Candidates	56
5.8	Out-of-Sample Behaviour: The Held-Out Year	56
5.9	A Negative Result: USD/JPY	58
5.10	Portfolio: The Seven Models Combined	58
5.10.1	Does the Result Depend on the Position Sizes?	59
6	Conclusion	61
6.1	Summary	61
6.2	Limitations	61
6.3	Future Work	62
	Bibliography	65

1 Introduction

The foreign exchange market is the largest financial market in the world, with an average turnover of 9.6 trillion US dollars per day in April 2025 [1], and it trades continuously from Sunday evening to Friday evening. This makes it a good target for automated trading. A program can act at any hour, the major currency pairs absorb retail order sizes without moving the price, and the price history is available down to individual quotes. The signal, however, is weak. Returns are close to unpredictable over short horizons [2], and the cost of trading is high enough that many strategies which look profitable in a simplified test lose money once those costs are charged.

This work applies Deep Reinforcement Learning (DRL) to that problem. One Deep Deterministic Policy Gradient (DDPG) [3] agent is trained for each of the seven major currency pairs and evaluated over eleven years of tick data, charging the spread quoted at the time of the trade and the commission of a retail account.

1.1 Motivation

Automated currency trading is usually built from fixed rules. Technical indicators are computed from recent prices, a signal is derived from them by comparison or crossover, and a separate layer decides the size of the position [4, 5]. These systems are cheap to run and easy to inspect, which is why they are common. Their parameters are set once and stay fixed, so the system does not adapt when the market changes. That is the first of the two difficulties this work deals with.

Currencies were chosen over equities for practical reasons [6]. The market trades continuously on weekdays, so a position is never carried through a daily close that cannot be traded. There are only seven major pairs, which makes it possible to test a design on all of them rather than on a sample selected by the author. Both directions are available without borrowing costs or short-selling restrictions, so a policy that sells is not penalised by mechanics unrelated to its signal. Every major pair also has the US dollar on one side, so the seven instruments are related without being interchangeable, and comparing results across them says something about the design rather than about one price series.

Reinforcement learning describes the same task without separating prediction from action. The agent observes the state of the market, chooses a position, and receives the profit or loss of that position as its reward [7]. It is never asked to forecast a price. It is asked to act, and the action is measured in money, which is how a trading system is judged in practice.

DDPG fits this task well. It is an actor-critic method for continuous action spaces [3], so the output of the actor can be used directly as a signed position: the sign gives the direction and the magnitude gives the exposure. Much of the published work on trading uses value-based methods such as the Deep Q-Network [8–10], which need the action to be reduced to a small discrete set, normally buy, hold and sell. Position sizing is then decided outside the learned policy, by rules that were never trained against the reward.

The second difficulty is cost, and it is independent of the learning algorithm. Results in this field are often reported on bar data, with the spread fixed or ignored and the broker commission left out [9, 11]. For a strategy that trades a few times per year this changes little. For a strategy that adjusts its position often it changes a lot, because the cost is paid on every adjustment while the profit per trade stays small. A system that is profitable without costs can lose money with them, and the difference does not appear unless the test charges what the account would pay [12]. The size of that difference is easy to underestimate. In this work, charging the broker commission on an otherwise identical run removed twenty percentage points of total return from a configuration that adjusted its position at every decision, and six percentage points once a threshold was added so that small adjustments were skipped. Both figures are reported in Section 5.

1.2 Problem Statement

Most published work on deep reinforcement learning for trading uses equity markets, daily bars, and a single instrument over a few years [9, 10, 13]. The work that does use currencies normally relies on a

discrete action space [14–16]. Trading costs are reported inconsistently: some studies charge both explicit and implicit costs, while others state returns with the spread fixed or omitted [11]. Two questions are left open. The first is whether one design works across several instruments instead of being fitted to one. The second is whether the reported performance survives realistic costs.

This work addresses both. The problem is to determine whether a single deep reinforcement learning design, trained separately for each instrument and otherwise unchanged, produces a profitable and risk-controlled policy on all seven major currency pairs, when the evaluation runs on tick data and charges the spread quoted at the time of the trade together with the commission of a swap-free raw-spread retail account.

Running one design across seven instruments also guards against a particular failure. A model that is adjusted until it performs on a single pair may be describing the history of that pair rather than a strategy, and nothing in its results will say which of the two it is [17]. Applying the same design unchanged to every major pair makes the difference visible, because a result that depends on the instrument will not hold across all of them.

Answering that question needs more than a learning algorithm. It needs an execution environment close enough to a real broker that the answer means something, and no such environment was available off the shelf. Building one took a substantial part of the effort behind this work, and its design is described in Section 4.

1.3 Objectives

The work was organised around five objectives.

- Build a trading framework that can backtest a strategy on tick data with broker-accurate costs and run the same strategy inside cTrader, both in its backtesting tab and on a live account, so that the tested system and the deployed system are the same code.
- Implement DDPG within that framework, with an observation built from price history and technical indicators, and with a continuous action read as a signed position size.
- Train and evaluate one agent per currency pair across the seven major pairs, over the same period and under the same cost model, so that any difference between pairs reflects the instrument rather than the setup.
- Determine whether the resulting performance comes from the policy or from exposure to the underlying currency movement.
- Combine the seven agents into a single equal-weight portfolio and measure what it does, since a trader would run them together rather than one at a time, and the behaviour of the combination is not implied by the seven results taken separately.

The first objective supports the other three. A result produced by a simplified simulator cannot be trusted or deployed, so the framework was built first and the learning experiments were run on top of it.

1.4 Contributions

The contributions of this work are the following.

- **A trading framework where one strategy implementation runs in three places.** The system has a Python library holding the strategy, the models and the market abstractions, and a C# component running inside cTrader [18] that connects the two through shared memory. The same strategy code runs in an offline backtest over stored market data, in the backtesting tab of cTrader, and on a live account, without modification.

- **A market database built from quote-level data.** The historical data was collected from cTrader through the same connector and stored locally. For the seven pairs studied here it holds about 1.66 billion quotes, occupying 295 GB, together with 33.0 million bars: 32.5 million at one minute, 547,835 at one hour, 23,386 daily and 1,050 monthly, all spanning January 2014 to August 2026. The engine selects the finest series available for each run, so the price path within a bar is reconstructed from the quotes themselves rather than interpolated.
- **A backtesting engine that charges what the account charges.** The engine steps on tick data instead of bar closes. The spread is the one quoted at the moment of the trade, read from the recorded quotes rather than assumed; the commission is computed from the traded volume at the rate of a raw-spread retail account; and stop and target levels are checked against the price path inside the bar rather than at its close. This reproduces the full cost structure of a swap-free raw-spread account, which is a product retail brokers actually offer: on such an account the spread and the commission are the two charges the trader pays, and both are applied here at the rates that account pays them. To our knowledge no published application of deep reinforcement learning to currency trading combines quote-level data, a measured spread and a real commission schedule in this way; Table 9 and Table 10 summarise what the reviewed studies report. It also means the returns reported in Section 5 are not directly comparable with results obtained on bar closes with a fixed or absent spread, and are lower than those results would be for the same policy.
- **DDPG applied to the seven major currency pairs under one protocol.** One agent was trained per pair, with the same architecture, observation design and reward. Each was evaluated between January 2015 and January 2026 under the same cost model, so the pairs can be compared with each other and the design is tested for generality rather than tuned to one instrument.
- **Positive and risk-controlled results on all seven pairs.** Total return over the eleven-year period ranged from +15.7% on USD/JPY to +226.6% on USD/CAD, and every pair stayed profitable when the evaluation was repeated from five different starting balances. The best pair on a risk-adjusted basis was USD/CHF, with a Sharpe ratio [19] of 0.80, a Sortino ratio [20] of 1.55, a Calmar ratio [21] of 0.49 and a maximum drawdown of 8.2% over 220 trades.
- **An attribution of the returns, including one negative result.** Regressing each model’s yearly return on the yearly movement of its own currency pair separates the return produced by the policy from the return produced by exposure to the market. Six of the seven pairs produced a positive intercept, and three of those had a market sensitivity close to zero. The exception is USD/JPY, whose model follows the underlying pair almost exactly and adds nothing to it. That case is reported here, together with the mechanism that causes it.

1.5 Structure

The rest of this document is organised as follows. Section 2 covers the material needed to follow the text: how the foreign exchange market works, how technical analysis summarises a price series, and how neural networks and reinforcement learning combine into the methods used here, ending with DDPG. Section 3 reviews the published applications of these methods to trading, grouped by market and by algorithm, and states what the existing work leaves open. Section 4 describes the system that was built: the framework and its connection to cTrader, the backtesting engine and its cost model, and the design of the observation, the action and the reward. Section 5 reports the evaluation protocol, the result for each of the seven pairs, how the agents behave while trading, what the trading cost, the robustness of the results to the entry path, the attribution of the returns into policy and market exposure, the screening that rejected weaker candidates, what happened in the year held back from training, and the outcome of running all seven together as a portfolio. Section 6 summarises what was achieved, states the limitations of the study and describes the work that follows from it.

This work draws on two fields that developed apart from each other. Trading supplies the market, the instruments, the costs and the way performance is measured. Reinforcement learning supplies the

learning algorithm and the way the problem is posed. Both are needed before the system itself can be described, and they are covered next.

2 Background

This section covers the material needed to follow the rest of the document. It starts with financial markets, and with the Forex market in particular, including how trading works there and what it costs. It then covers neural networks and reinforcement learning, the two machine learning fields this work draws on, and ends with Deep Reinforcement Learning, where the two meet, and with the algorithm used here.

2.1 Financial Markets and Trading

Financial markets move capital between those who hold it and those who need it. They allow participants to trade assets, hedge risk and raise funds, and each type of market serves a different purpose. Table 1 summarises the main ones. The markets are connected, so a movement in one is often transmitted to the others and none of them is analysed in isolation. Prices are set by the participants themselves, from private investors to banks and investment funds, while regulators set the rules under which those participants operate. Electronic platforms and algorithmic execution have widened access and increased the speed at which prices adjust [5].

Market	Economic Role	Asset Examples
Stock Market	Capital raising through company shares trading.	Stocks, Equities
Bond Market	Debt financing by entities borrowing from investors.	Government and Corporate Bonds
Commodities Market	Trading of essential raw materials and agricultural goods.	Oil, Gold, Wheat
Forex Market	Currency exchange for international trade.	USD, EUR, JPY, GBP
Derivatives Market	Trading contracts for hedging or speculation.	Futures, Options, Swaps
Indices Market	Aggregating and tracking market or sector performance.	S&P 500, NASDAQ
Cryptocurrency Market	Exchanging digital currencies.	Bitcoin, Ethereum

Table 1: Summary of Financial Markets and respective Economic Role [5].

2.1.1 Forex Market: Characteristics and Significance

The Foreign Exchange market, or Forex market, is where currencies are exchanged. It is the largest financial market by turnover, averaging 9.6 trillion US dollars per day in April 2025, a 28% increase on the 7.5 trillion recorded three years earlier, and it operates continuously across time zones because the banks and institutions that quote its prices are spread worldwide. Spot transactions, the instrument traded in this work, account for 3 trillion of that daily total. The US Dollar sits on one side of 89.2% of all trades, and the ten most traded currency pairs all involve it [1]. Prices respond to interest rate decisions, economic releases and political events. Participants normally trade with leverage, which allows a position much larger than the deposited capital and scales profits and losses in the same proportion. Depth, continuous operation, sensitivity to news and the routine use of leverage are what make the market suitable for automated trading and hard to model at the same time. Its behaviour is noisy and non-stationary, which is why adaptive and data-driven strategies are of interest [6].

2.1.2 Forex Trading: Fundamental Concepts and Operation

Trading currency pairs involves buying one currency and simultaneously selling another. Buying the EUR/USD pair means purchasing Euros and selling US Dollars, in the expectation that the Euro will strengthen against the US Dollar. This work uses the seven major currency pairs: EUR/USD, USD/JPY, GBP/USD, AUD/USD, USD/CHF, NZD/USD and USD/CAD. They are built from the European Euro (EUR), US Dollar (USD), Japanese Yen (JPY), Great Britain Pound (GBP), Australian Dollar (AUD), Swiss Franc (CHF) and New Zealand Dollar (NZD). Every one of them has the US Dollar on one side,

and deciding whether to buy or sell a pair amounts to predicting the strength of one currency relative to the other [6].

A position is the exposure held in a pair. A long position profits when the pair rises and loses when it falls; a short position does the opposite. Because a currency pair is a ratio between two currencies, selling is as natural as buying and requires no borrowing, which is not the case in equity markets. Positions are opened and closed with orders. A market order executes immediately at the price currently quoted. A limit order executes only at a specified price or better, and a stop order executes once the price passes a specified level. Two stop orders are commonly attached to an open position: a stop loss, which closes it once losses reach a chosen level, and a take profit, which closes it once a target is reached.

Forex trading occurs in a decentralised market that operates globally. There is no exchange where all transactions take place. Instead, a network of market makers, primarily banks (known as the interbank market) and brokers, handles trades. Interbank transactions involve large volumes and are dominated by institutions such as multinational corporations and major hedge funds. Retail traders participate through brokers with access to that network, who act as intermediaries. Market makers and brokers quote two prices at all times: the bid price, at which they are willing to buy the pair, and the ask price, at which they are willing to sell it. The difference between them is the spread. The absence of a central venue might suggest a lack of regulation, but competition among market makers keeps quoted prices close to one another and spreads narrow [6].

Traders read price through charts, and the candlestick chart is the standard form. Each chart is made up of candlesticks, which are structures that represent price movements over a time span that can range from minutes to years. A tick is a single update to the quoted price, published whenever a market maker revises its bid or its ask. Each candlestick, named for its resemblance to a double-sided candle, provides information on the opening price (O), the highest price (H), the lowest price (L), and the closing price (C) for a specific period of time (see Figure 1). The colour of the candlestick represents the price movement, with lighter shades indicating an increase and darker shades indicating a decrease. This allows traders to gain a fast understanding of the direction of the market just by looking at the chart. Candlestick charts do not show volume (V), which is normally plotted separately below the price. Because Forex has no central venue, no consolidated record of traded size exists, so brokers report tick volume instead: the number of quote updates in the period. That is the measure used throughout this work [22].

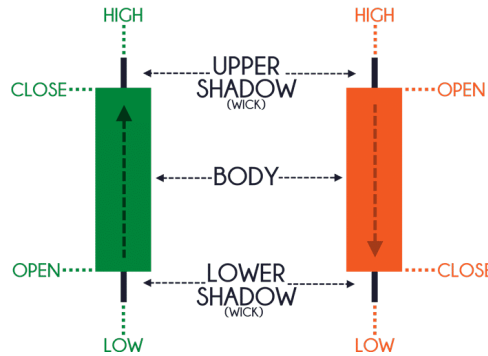


Figure 1: Candlesticks encoding upward (bullish) and downward (bearish) movements respectively [23].

This work uses tick data as its primary record, since it is the finest resolution the broker provides, and derives the opening, highest, lowest and closing prices together with the tick volume, or OHLCV data for short, whenever a coarser view is required [22]. Indicators computed from that data are used to give the model additional context, as described next.

2.1.3 Forex Costs: Spreads, Commissions and Overnight Financing

Trading is not free, and the charges are large relative to the profit available on a single position. Three of them apply in Forex, and they are charged independently of one another. Prices are quoted

in units that follow from the pair. A pip is the fourth decimal place of the quote for most pairs, or the second for pairs quoted against the Japanese Yen. A point is one tenth of a pip and is the smallest increment a broker quotes. Position size is expressed in units of the base currency, where one standard lot is 100,000 units. For a position of volume q at price p , the notional value in the quote currency is

$$V = q \cdot p \quad (1)$$

The **spread** is the first charge and is paid implicitly. A position is opened at the ask and closed at the bid, so the round trip crosses the spread once. For a position of volume q , the cost is

$$C_{spread} = (p_{ask} - p_{bid}) \cdot q \quad (2)$$

The spread is not constant. It widens when liquidity falls, around economic releases and at the daily rollover, so a strategy tested with a fixed spread is tested against a market that does not exist.

The **commission** is the second charge and is explicit. Accounts that quote raw interbank spreads recover their margin through a commission charged on the traded volume rather than through a wider spread. Expressed as a number of points c , with s_{point} the size of one point, the cost of a position of volume q is

$$C_{commission} = c \cdot s_{point} \cdot q \quad (3)$$

The spread and the commission are separate charges and both are paid. Treating the spread as if it already included the commission understates the cost of trading on a raw-spread account.

The **swap**, or overnight financing, is the third charge and reflects the interest rate differential between the two currencies of the pair. It is applied when a position is still open at the daily rollover, and is charged at a multiple on one day of the week to account for the weekend. Depending on the direction of the position and the sign of the differential, it can be a cost or a credit. Some account types, offered so that clients can avoid interest for religious reasons, remove it entirely; these are known as swap-free accounts. Finally, leverage determines how much capital a position requires. With leverage L , the margin committed to a position of notional value V is

$$M = \frac{V}{L} \quad (4)$$

Leverage does not change the profit or loss of a position, only the capital reserved against it, so it acts on the return of the account rather than on the return of the trade. These charges do not apply uniformly across account types, and the choice of account is therefore part of the design of a strategy rather than an administrative detail. A raw-spread account quotes the interbank spread with no markup and recovers its margin through the commission, which makes the total cost of a position lower and, more usefully, explicit: the implicit part shrinks to the market spread and the rest is a published rate. A swap-free account charges no overnight financing at all. The two together suit a strategy that holds positions for days rather than seconds, because such a strategy pays the spread and the commission only a few times per position but would otherwise pay financing on every night the position stays open. This work evaluates on an account that is both, so the spread and the commission are the only charges applied, and the financing term is genuinely zero rather than estimated.

2.1.4 Financial Analysis: Main Types and Formulas

Representing the data is one half of the problem; the other is analysing it to decide what to trade. Financial analysis is generally divided into three types [24]:

- **Fundamental Analysis:** examines economic factors that affect currency prices. This includes analysis of GDP growth, employment rates, interest rates, and monetary policies, which help predict currency trends based on a nation's economic health. However, the qualitative nature of these data poses challenges to computational models that prefer structured numerical input.

- **Sentiment Analysis:** seeks to quantify the mood of the market by analysing news, market commentary, and similar qualitative data. It aims to measure how collective emotions impact market trends, although its complexity and need for advanced natural language processing make it less straightforward for model integration.
- **Technical Analysis:** involves analysing previous OHLCV data to predict future market movements. It operates on the principle that past market behaviour will repeat itself, and thus identifying past patterns or metrics on current data can predict future movements. It is preferred for its quantitative results, which can be used by complex machine learning models.

Of the three, this work uses technical analysis, because it produces numerical series directly from price data and therefore needs no additional interpretation before a model can consume it. Technical analysis divides into indicators and candlestick patterns. Indicators are metrics created mainly by mathematicians and economists, calculated from historical market data, that offer a quantitative view of market dynamics. They are generally categorised into four main types [24]:

- **Trend Indicators:** Used to identify the prevailing direction of the market by filtering out market noise. Refer to Table 2 for more details on frequently used indicators.
- **Momentum Indicators:** Used to capture the momentum of a given trend, whether it is strong or weak, and if a reversal is expected. Refer to Table 3 for more details on frequently used indicators.
- **Volume Indicators:** Used to provide a perspective on the trading volume, providing clues about the strength of a price movement. Refer to Table 4 for more details on frequently used indicators.
- **Volatility Indicators:** Used to identify the level of uncertainty or risk associated with the volatility of the price movement. Refer to Table 5 for more details on frequently used indicators.

Indicator Name	Formula	Range
Simple Moving Average	$SMA(N)_t = \frac{1}{N} \sum_{i=1}^N Price_{t-N+i}$	-
Exponential Moving Average	$EMA(N)_t = \frac{2}{N+1} \times Price_t + (1 - \frac{2}{N+1}) \times EMA(N)_{t-1}$	-
Weighted Moving Average	$WMA(N)_t = \frac{2}{N \times (N+1)} \sum_{i=1}^N i \times Price_{t-N+i}$	-
Double Exponential Moving Average	$DEMA(N)_t = 2 \times EMA(N)_t - EMA_2(N)_t$	-
Triple Exponential Moving Average	$TEMA(N)_t = 3 \times [EMA(N)_t - EMA_2(N)_t] + EMA_3(N)_t$	-
Hull Moving Average	$HMA(N)_t = WMA(\sqrt{N}) \left[2 \times WMA\left(\frac{N}{2}\right)_t - WMA(N)_t \right]$	-
Triangular Moving Average	$TRIMA(N)_t = \frac{1}{N} \sum_{i=1}^N SMA(N)_{t-N+i}$	-
Efficiency Ratio	$ER(N)_t = \frac{ Price_t - Price_{t-N} }{\sum_{i=1}^N Price_{t-i+1} - Price_{t-i} }$	[0, 1]
Kaufman Adaptive Moving Average	$KAMA(N)_t = KAMA(N)_{t-1} + SC \times (Price_t - KAMA(N)_{t-1})$ $SC = [ER(N)_t \times (\phi_{short} - \phi_{long}) + \phi_{long}]^2, \phi_{short} = \frac{2}{N_{short}+1}, \phi_{long} = \frac{2}{N_{long}+1}$ where typically $N_{short} = 2$ and $N_{long} = 30$	-

Table 2: Summary of frequently used Trend Technical Indicators [25].

The four tables survey indicators in common use rather than the contents of any one system. The framework built for this work implements the moving average family and the crossovers between its members, the Rate of Change, the Moving Average Convergence Divergence, the Average True Range, the Realised Volatility and the Efficiency Ratio. It implements no volume indicators, because the tick volume a Forex broker reports counts quote updates rather than traded size, which limits what those indicators can measure. Ten instances of the implemented set form the observation the agent receives,

Indicator Name	Formula	Range
Price Momentum	$PM_t = \frac{Price_t - Price_{t-1}}{Price_{t-1}}$	-
Rate of Change	$ROC(N)_t = \ln\left(\frac{Price_t}{Price_{t-N}}\right)$	-
Price Acceleration	$PA_t = \frac{\text{Short Linear Regression Coefficient}}{\text{Long Linear Regression Coefficient}} = \frac{\beta_{\text{short}}}{\beta_{\text{long}}}$	-
Relative Strength Index	$RSI(N)_t = 100 - \frac{100}{1 + \frac{\text{Average Gain}_t}{\text{Average Loss}_t}}$ $\text{Average Gain}_t = \frac{1}{N} \sum_{i=1}^N \max(Price_i - Price_{i-1}, 0)$ $\text{Average Loss}_t = \frac{1}{N} \sum_{i=1}^N \max(Price_{i-1} - Price_i, 0)$	[0, 100]
Aroon Oscillator	$Aroon(N)_t = Aroon\ Up(N)_t - Aroon\ Down(N)_t$ $Aroon\ Up(N)_t = \left(\frac{N - \text{Periods Since Highest High in } N}{N}\right) \times 100$ $Aroon\ Down(N)_t = \left(\frac{N - \text{Periods Since Lowest Low in } N}{N}\right) \times 100$	[-100, 100]
Balance of Power	$BOP_t = \frac{Close_t - Open_t}{High_t - Low_t}$	[-1, 1]
Commodity Channel Index	$CCI(N)_t = \frac{TP_t - SMA_{TP}(N)_t}{0.015 \times \text{Mean Deviation}(N)_t}$ $TP_t = \frac{High_t + Low_t + Close_t}{3}$ $\text{Mean Deviation}(N)_t = \frac{1}{N} \sum_{i=1}^N TP_i - SMA_{TP}(N)_t $	-
Moving Average Convergence Divergence	$MACD_t = EMA(N_{\text{short}})_t - EMA(N_{\text{long}})_t$ where typically $N_{\text{short}} = 12$ and $N_{\text{long}} = 26$	-
Stochastic RSI	$StochRSI(N)_t = \frac{RSI(N)_t - \min(RSI(N))}{\max(RSI(N)) - \min(RSI(N))}$	[0, 1]
Stochastic Oscillator	$Stoch\ \%K(N)_t = \frac{Close_t - LL(N)_t}{HH(N)_t - LL(N)_t} \times 100$ $Stoch\ \%D(M)_t = SMA_{Stoch\ \%K(N)}(M)_t$ $HH(N)_t = \max_{i=t-N+1}^t (High_i)$, $LL(N)_t = \min_{i=t-N+1}^t (Low_i)$	[0, 100]
Ultimate Oscillator	$UO_t = \frac{4 \times Avg_t(N_1) + 2 \times Avg_t(N_2) + Avg_t(N_3)}{4 + 2 + 1} \times 100$ $Avg_t(N) = \frac{\sum_{i=0}^{N-1} BP_{t-i}}{\sum_{i=0}^{N-1} TR_{t-i}}$ $BP_t = Close_t - \min(Close_{t-1}, Low_t)$ $TR_t = \max(Close_{t-1}, High_t) - \min(Close_{t-1}, Low_t)$ where typically $N_1 = 7$, $N_2 = 14$, and $N_3 = 28$	[0, 100]
Williams %R	$Williams\ \%R(N)_t = \frac{HH(N)_t - Close_t}{HH(N)_t - LL(N)_t} \times -100$	[-100, 0]

Table 3: Summary of frequently used Momentum Technical Indicators [25].

Indicator Name	Formula	Range
Chaikin Accumulation/Distribution Line	$AD_t = AD_{t-1} + MFV_t$ $MFV_t = \frac{(Close_t - Low_t) - (High_t - Close_t)}{High_t - Low_t} \times Volume_t$	-
Chaikin Accumulation/Distribution Oscillator	$ADOSC_t = EMA_{AD}(N_{\text{short}})_t - EMA_{AD}(N_{\text{long}})_t$ where typically $N_{\text{short}} = 3$ and $N_{\text{long}} = 10$	-
On-Balance Volume	$OBV_t = \begin{cases} OBV_{t-1} + Volume_t & \text{if } Price_t > Price_{t-1} \\ OBV_{t-1} - Volume_t & \text{if } Price_t < Price_{t-1} \\ OBV_{t-1} & \text{otherwise} \end{cases}$	-

Table 4: Summary of frequently used Volume Technical Indicators [25].

Indicator Name	Formula	Range
Standard Deviation	$SD(N)_t = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (\text{Price}_i - \overline{\text{Price}})^2}$	-
Realised Volatility	$RV(N)_t = \sqrt{\sigma_t^2}$, where $\sigma_t^2 = \alpha \times r_t^2 + (1 - \alpha) \times \sigma_{t-1}^2$, $r_t = \ln\left(\frac{\text{Price}_t}{\text{Price}_{t-1}}\right)$ and $\alpha = \frac{1}{N}$	-
Bollinger Bands	Upper $BB(N, k)_t = SMA(N)_t + k \times SD(N)_t$ Lower $BB(N, k)_t = SMA(N)_t - k \times SD(N)_t$ where typically $k = 2$	-
Average True Range	$ATR(N)_t = \frac{(N-1) \times ATR(N)_{t-1} + TRANGE_t}{N}$ $TRANGE_t = \max(\text{High}_t - \text{Low}_t, \text{High}_t - \text{Close}_{t-1} , \text{Low}_t - \text{Close}_{t-1})$	-
Normalised Average True Range	$NATR(N)_t = \frac{ATR(N)_t}{\text{Close}_t} \times 100$	-
Chaikin Volatility Index	$CVI(N)_t = \frac{EMA_{HL}(N)_t - EMA_{HL}(N)_{t-N}}{EMA_{HL}(N)_{t-N}} \times 100$ $EMA_{HL}(N)_t = EMA(N)$ applied to $(\text{High}_t - \text{Low}_t)$	-

Table 5: Summary of frequently used Volatility Technical Indicators [25].

listed in Section 4. Candlestick patterns take a different approach. Rather than deriving a value through a calculation, they describe recognisable shapes formed by one or more candlesticks that traders associate with a change in behaviour [22]. Such patterns can be encoded numerically and supplied to a model in the same way as an indicator [25], although this work does not use them: the observation is built from indicators alone.

2.1.5 Performance Measurement: Returns, Risk and Ratios

A trading strategy is judged by what it earns and by the risk taken to earn it, so a single return figure is not enough to compare two strategies. The measures below are the ones reported in Section 5. They are defined on an equity series and therefore apply unchanged to a portfolio of several strategies, computed on the combined equity rather than on each strategy separately, which matters because a portfolio's risk depends on how its components move together and not only on their individual risks.

The total return over a period compares final and initial account equity E :

$$R = \frac{E_{final} - E_{initial}}{E_{initial}} \quad (5)$$

Because periods differ in length, returns are annualised before they are compared. With a duration Δt in seconds and a year taken as 365 days,

$$R_{annual} = (1 + R)^{\frac{365 \cdot 86400}{\Delta t}} - 1 \quad (6)$$

Risk is measured in two ways. The first is volatility, the standard deviation of returns, annualised by the rate at which the strategy trades. The second is drawdown. At any moment the drawdown is the distance between current equity and the highest equity reached so far:

$$D(t) = \frac{E(t) - \max_{s \leq t} E(s)}{\max_{s \leq t} E(s)} \quad (7)$$

The maximum drawdown is the worst value of $D(t)$ over the period and describes the largest loss an investor would have experienced from a peak. The mean drawdown averages $D(t)$ instead, and describes how far below its peak the account sits on a typical day rather than at its worst moment. Four ratios combine return and risk. All of them divide an annualised excess return by a measure of risk, differing only in which measure they use. The magnitude of that measure is taken, since a drawdown is negative under the definition above. Writing R_f for the risk-free rate,

$$\text{Ratio} = \frac{R_{\text{annual}} - R_f}{|\text{risk}|} \quad (8)$$

The **Sharpe ratio** [19] uses annualised volatility, and therefore penalises upward and downward movement equally. The **Sortino ratio** [20] replaces it with downside volatility, computed from negative returns only, on the argument that an investor does not object to upside variability. The **Calmar ratio** uses the maximum drawdown, so it measures return against the worst loss suffered. The **Sterling ratio** uses the mean drawdown instead, which makes it less sensitive to a single bad episode and more representative of the typical experience of holding the strategy. Both Calmar and Sterling appear in the literature with several definitions, and the ones used here follow [21]. The mean drawdown is smaller than the maximum by construction, so for a strategy with a positive excess return the Sterling ratio is the larger of the two, and the ordering reverses when the excess return is negative.

2.1.6 Algorithmic and Quantitative Trading

Financial markets are increasingly dominated by algorithms. Algorithmic trading uses computer programs to execute trades faster and in greater volume than a human can, by analysing large datasets, interpreting market conditions and acting on defined criteria. It is applied to pre-trade analysis, signal generation and trade execution, and it is what allows institutions to run complex strategies at scale. Quantitative trading is a subfield of it, in which mathematical and statistical models are used to find opportunities in market data through historical analysis and machine learning. The approach relies on systematic rules rather than discretionary judgment, and demands knowledge of both finance and mathematics [4]. Together they have changed how markets operate, introducing practices such as high-frequency trading and shortening the time in which prices react to information. They have also made machine learning central to the field: neural networks for extracting structure from market data, and reinforcement learning for deciding what to do with it [25]. Those two are the subject of the sections that follow.

2.2 Neural Networks

Machine learning covers algorithms that find patterns in data and act on them without being programmed with the rules in advance. Tom Mitchell states the idea as follows [26]:

"A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E."

The definition is useful here because it describes exactly what a trading agent does: the task is choosing a position, the performance measure is the money that position makes, and the experience is the market data it has already seen. Neural networks, the subject of this section, are the class of models used to represent that behaviour.

2.2.1 Fundamentals and History

Neural Networks (NN), named for their similarity to the structure of the human brain, are algorithms designed for pattern recognition. In the human brain, a neuron receives signals through structures called dendrites, processes these signals in the nucleus, and passes the resultant processed signal down its axon to the dendrites of adjacent neurons. Similarly, in a neural network, each node represents an artificial neuron that receives input, processes it through a mathematical function, and passes the output to the subsequent layer of nodes. In both cases, the strength and nature of the connections between neurons determine the behaviour of the network [27]. Refer to Figure 2 for a visual representation of the concepts explained above.

The concept of neural networks dates back to the 1950s with the perceptron, a simple model of a biological neuron developed by Frank Rosenblatt [29]. The perceptron marked the early stages of NN

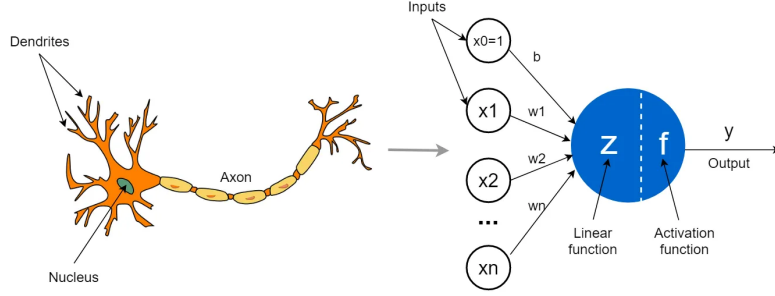


Figure 2: Human Neuron compared to an Artificial Neuron [28].

research, but was limited to linearly separable datasets, leading to some periods of reduced interest in the field, known as "AI winters". The 1980s brought a significant breakthrough with the backpropagation algorithm, which enabled neural networks to learn patterns in non-linearly separable datasets using complex and multilayered models [30].

2.2.2 Key Concepts and Feed-Forward

In a neural network, the feed-forward computation within each layer l is based on the data processed by its predecessor. For the first layer, this input x is the raw data itself, while for subsequent layers, it is the output of the previous layer $h^{(l-1)}(x)$. The computation in each layer can be formalised as [27]:

$$z^{(l)}(x) = W^{(l)}h^{(l-1)}(x) + b^{(l)} \quad (9)$$

Here, the weight matrix $W^{(l)} \in \mathbb{R}^{K_l \times K_{l-1}}$ and the bias vector $b^{(l)} \in \mathbb{R}^{K_l}$ are required for the computation at layer l , with K_l representing the number of neurons or units in layer l , and K_{l-1} indicating the number of neurons in the preceding layer $l-1$. In particular, $W^{(1)} \in \mathbb{R}^{K_1 \times D}$ for the first layer, where D is the size of the input data. The activation function g is subsequently applied to the pre-activation $z^{(l)}(x) \in \mathbb{R}^{K_l}$ to produce the activated output $h^{(l)}(x) \in \mathbb{R}^{K_l}$ [27]:

$$h^{(l)}(x) = g(z^{(l)}(x)) \quad (10)$$

The choice of activation function matters in two different places. In the hidden layers a non-linear function is required, because a stack of linear layers collapses into a single linear transformation and the network would gain nothing from its depth. In the output layer the function is dictated by the task: a bounded function when the output has to stay within a range, and no activation at all when it does not [27]. Refer to Table 6 for a summary of the most common activation functions.

Activation Function	Formula	Description
Linear	$a(x) = x$	Suitable for regression tasks.
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	Maps inputs to (0, 1), for binary classification.
Hyperbolic Tangent	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	Maps inputs to (-1, 1), offering richer gradients than sigmoid.
ReLU	$\text{ReLU}(x) = \max(0, x)$	Encourages sparse activation, helps with gradient flow.
Softmax	$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}}$	Turns logits into probabilities, for multi-class classification.

Table 6: Summary of the most common Activation Functions [27].

2.2.3 Learning using Backpropagation

Neural networks learn by iteratively adjusting weights and biases to minimise the error between predicted outputs and actual target values. This learning process is guided by a loss function that quantifies the model's error. The goal is to optimise the network by finding parameters that minimise this loss function. Different problems require different loss functions, such as mean squared error for regression tasks and cross-entropy for classification [27]. Refer to Table 7 for a detailed overview of their formulas.

Loss Function	Formula	Application
Mean Squared Error (MSE)	$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Regression
Negative Log-Likelihood (Cross-Entropy)	$-\sum_{i=1}^n y_i \log(\hat{y}_i)$	Classification

Table 7: Summary of the most common Loss Functions [27].

The core algorithm used to train neural networks is known as backpropagation [30]. It computes the gradient of the loss function with respect to the network's weights and biases, which gives the direction of steepest increase in the loss. Refer to Algorithm 1 for a detailed overview of how these gradients are calculated. Gradient descent then updates the weights and biases in the opposite direction, reducing the loss. Its variants, outlined with the respective formulas in Table 8, address different data and computational needs. Batch gradient descent uses the entire dataset for each update, which is stable but expensive on large datasets. Stochastic gradient descent updates the parameters after every data point, which is faster but noisier. Mini-batch gradient descent updates on subsets of the data, which is the compromise used in practice [27].

Algorithm 1 Gradient of the Loss Function [27].

Require: f : network function, θ : network parameters, x : input data, y : true label, l : number of layers, L : loss function
 Compute the output of the network $\hat{y} = f(x; \theta)$
 Compute the gradient of the loss function with respect to the output layer before activation:
 $\nabla_{z^{(l+1)}} L(f(x; \theta), y) = \frac{\partial L(f(x; \theta), y)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(l+1)}}$
for $\ell = l + 1$ to 1 **do**
 Compute gradients of hidden layer parameters:
 $\nabla_{W^{(\ell)}} L(f(x; \theta), y) = \nabla_{z^{(\ell)}} L(f(x; \theta), y) \cdot h^{(\ell-1)T}$
 $\nabla_{b^{(\ell)}} L(f(x; \theta), y) = \nabla_{z^{(\ell)}} L(f(x; \theta), y)$
 Compute gradient of previous hidden layer:
 $\nabla_{h^{(\ell-1)}} L(f(x; \theta), y) = W^{(\ell)T} \cdot \nabla_{z^{(\ell)}} L(f(x; \theta), y)$
 Compute gradient of previous hidden layer (before activation):
 $\nabla_{z^{(\ell-1)}} L(f(x; \theta), y) = \nabla_{h^{(\ell-1)}} L(f(x; \theta), y) \odot g'(z^{(\ell-1)})$
end for

In Table 8, the learning rate η is a hyperparameter that controls the size of each update. A rate that is too large overshoots the minimum, and one that is too small converges slowly and can settle in a local minimum. Methods that adapt η during training address both problems by changing the step size as the optimisation progresses. Normalising or standardising the input data serves the same end, since features on very different scales produce gradients on very different scales [27].

2.2.4 Bias-Variance Trade-off and Model Generalisation

The generalisation power of a neural network depends on finding a balance between bias and variance. Bias leads to underfitting, where the model overlooks underlying trends due to overly simplistic assumptions, while variance leads to overfitting, where the model interprets noise as a signal due to

Algorithm	Formula	Description
Batch Gradient Descent	$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; X, y)$	Updates parameters using all data
Mini-Batch Gradient Descent	$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; X^{(i:i+n)}, y^{(i:i+n)})$	Updates parameters using subsets of data
Stochastic Gradient Descent	$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}, y^{(i)})$	Updates parameters per data point

Table 8: Summary of the most common Gradient Descent variants [27].

excessive complexity. Both result in poor performance on new data, which matters in Forex trading because the signal being sought is weak and the noise around it is large. Cross-validation is commonly employed to evaluate a model’s generalisation power on independent training, validation and testing datasets. Regularisation techniques such as L1 and L2 moderate model complexity by adding penalties to the loss function, and dimensionality reduction methods such as principal component analysis reduce the number of inputs the model has to fit. Together these techniques raise the chance of producing a model that performs on out-of-sample data [27]. The approach taken in this work is to divide the period into training, validation and testing windows, described in Section 4, and to let performance on the validation window decide which model is kept, rather than to control complexity through a penalty added to the loss.

2.2.5 Architectures for Sequential Data

The network described so far is feed-forward: every input is presented at once and the output depends only on that input. Financial data is a sequence, so an obvious question is whether a network that reads a sequence directly would do better.

Two families do that. A recurrent neural network keeps an internal state that is carried from one step to the next, so an observation is interpreted in the light of the ones before it. The long short-term memory network is the variant used in practice, because it adds gates that control what the state keeps and what it discards, which prevents the gradient from vanishing as it is propagated back over many steps [27]. Attention-based networks reach the same goal differently, by letting the network weigh every position in a window directly rather than passing information along a chain.

Much of the work reviewed in Section 3 uses recurrence, and several of those studies report that it helped. This work does not, and the reason is a design decision rather than an oversight. Recurrence is one way to give a network access to history; another is to compute that history explicitly and present it as part of the input. The observation described in Section 4 takes the second route: it carries momentum and moving-average readings over horizons from one day to a year, so the history a recurrent state would have to learn to summarise is instead measured directly and handed to the network.

That choice has three consequences worth stating. A feed-forward network over engineered features has far fewer parameters than a recurrent one over raw prices, which matters when the signal is weak and the sample is one instrument over eleven years. It is also faster to train, and training speed decided how many independent runs could be afforded, which turned out to matter more for the quality of the final result than any architectural gain would have. And it keeps the input interpretable: each feature is a named quantity with a known meaning, so when a policy behaves strangely the observation can be inspected rather than guessed at.

The cost of that choice is real and is not hidden. A recurrent network can in principle discover a summary of the past that no fixed set of indicators contains, and this work forgoes that possibility. Whether it would have helped here is untested, and it is listed among the extensions in Section 6.3.

2.2.6 Universal Approximation Theorem

The universal approximation theorem, established by Hornik, Stinchcombe and White in 1989, can be stated as follows [31]:

A neural network with one hidden layer and a linear output can approximate any continuous function arbitrarily well, given enough hidden units.

The theorem establishes neural networks as universal function approximators, which is what justifies their use here. It says that a network of sufficient size can represent the function mapping market observations to a trading decision. It says nothing about whether such a function can be found from data, or whether it exists in a form stable enough to be useful, and those questions are the subject of the reinforcement learning methods covered next.

2.3 Reinforcement Learning

Reinforcement Learning (RL) is the branch of machine learning concerned with an agent that interacts with an environment and learns which decisions to make. It differs from supervised learning in that no correct answer is supplied for any given situation. The agent acts, observes the consequence, and adjusts its behaviour according to the reward it receives. Learning therefore happens through trial and error rather than from labelled examples, which suits problems where the right action is not known in advance but its outcome can be measured. Robotics and finance are two fields where that description applies.

2.3.1 Fundamentals and History

Reinforcement Learning is a field of study that emerged from the combination of many other fields like psychology, computer science, and control theory, and it was inspired by the way humans and animals learn from their environment. RL involves an agent that makes decisions, receiving rewards or penalties based on its actions, aiming to maximise positive outcomes over time. This process is deeply rooted in the concept of operant conditioning, a type of learning studied by psychologists such as B. F. Skinner [32], where behaviour is shaped by its consequences. Markov Decision Processes (MDPs) were formalised in the late 1950s as a way to model decision making where outcomes are partly random and partly under the control of the decision maker. This became the mathematical backbone of RL, as it allowed the development of algorithms that could learn optimal strategies in complex environments. One of the major breakthroughs in RL was the introduction of Q-Learning in the late 1980s, an algorithm capable of learning the value of actions in different states, helping agents make informed decisions without needing a model of the environment. This, along with Temporal Difference Learning, set the stage for the application of RL to a wide range of problems, from playing video games at a superhuman level to optimising financial trading strategies [7].

2.3.2 Key Concepts and Agent-Environment Interaction

Reinforcement Learning is about how an agent interacts with its environment. This interaction involves states, actions and rewards. The state S_t represents the situation of the agent at that time instant. When the agent observes S_t it chooses an action A_t . This action causes the environment to transition to a state S_{t+1} , providing feedback in the form of a reward R_{t+1} . Refer to Figure 3 for a visual representation of this cyclical interaction [7].

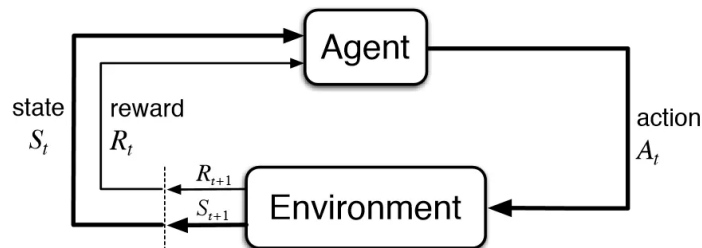


Figure 3: Interaction between the agent and the environment in a Reinforcement Learning setting [7].

The objective of the agent is to maximise the return, which is the discounted sum of the rewards it collects from time t onwards [7]:

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (11)$$

Here, γ represents the discount factor, indicating the degree to which future rewards are considered important relative to immediate ones. A policy π directs the agent's actions, which can be deterministic, assigning a specific action to each state, or stochastic, assigning a probability distribution over actions. Balancing between exploration, the act of trying new actions to discover their effects, and exploitation, the act of choosing actions that are known to yield high reward, is a fundamental challenge in RL. A common approach is the ϵ -greedy policy, which mostly selects the best-known action but occasionally selects a random one [7]. This method requires a discrete set of actions, since it works by picking an alternative at random. Where the action is continuous, as it is in this work, exploration is instead obtained by adding noise to the action the policy returns, an approach described in Section 2.4 together with the algorithm that uses it.

The fundamental concept behind Reinforcement Learning lies in the Markov Decision Process (MDP) framework, defined by the tuple (S, A, P, R, γ) . Here, S represents all possible states in the environment, A denotes the actions the agent can take, and P is the transition probability, indicating the likelihood of moving from state s_t to state s_{t+1} after taking action a . R is the reward function, which gives the reward obtained for that transition, and γ is the discount factor. MDPs rely on the Markov property, which requires that the next state depend only on the current state and action and not on any earlier ones [7]:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1}|s_t, a_t) \quad (12)$$

The reward function matters because it is where the objective of the task is expressed. It tells the agent what to achieve without telling it how, and any behaviour that raises the reward is a behaviour the agent will adopt. In a trading setting the reward is normally derived from profit and loss, so the agent is directed towards profitable positions without being told which positions those are [7].

2.3.3 Value Functions and Q-Functions

The value function for a policy π , denoted as $V^\pi(s)$, represents the expected return when starting in state s and following policy π afterwards. It is formulated as follows [7]:

$$V^\pi(s) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, \pi \right] \quad (13)$$

Here, γ is the discount factor, which weighs the importance of future rewards compared to immediate rewards, and $\mathbb{E}$ denotes the expected value, reflecting the average of all possible outcomes weighted by their probabilities. The Q-function for a policy π , denoted as $Q^\pi(s, a)$, represents the expected return for taking an action a in state s and following policy π afterwards. It is formulated as follows [7]:

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a, \pi \right] \quad (14)$$

The Bellman equations, named after Richard Bellman, who first formulated them, provide a recursive decomposition for the total cumulative reward an agent can expect to receive starting from a particular state or state-action pair. These equations are fundamental to dynamic programming techniques in RL, as they express the value of a state, or of a state-action pair, in terms of the values of the states that follow it. That recursion is what allows the estimated values to be improved iteratively, which is the basis of convergence for most RL algorithms. This is expressed as follows [7]:

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')] \quad (15)$$

$$Q^\pi(s, a) = \sum_{s' \in S} P(s'|s, a) \left[R(s, a, s') + \gamma \sum_{a' \in A} \pi(a'|s') Q^\pi(s', a') \right] \quad (16)$$

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) Q^\pi(s, a) \quad (17)$$

The relationship pointed in Equation 17 indicates that the value of a state under policy π is the expected value of the Q-function over all possible actions, weighted by the policy's action probabilities. For optimal policies, the optimal value function $V^*(s)$ and the optimal Q-function $Q^*(s, a)$ represent the maximum possible returns achievable from any state or state-action pair, respectively. Following the Bellman equation, it is possible to obtain [7]:

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')] \quad (18)$$

$$Q^*(s, a) = \sum_{s' \in S} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a' \in A} Q^*(s', a') \right] \quad (19)$$

$$V^*(s) = \max_{a \in A} Q^*(s, a) \quad (20)$$

Through the iterative application of these equations, RL algorithms gradually approximate the value and Q-functions, guiding the agent towards the optimal policy that maximises the expected cumulative reward [7].

2.3.4 Methodological Variants and Learning Methods

The field of reinforcement learning can be divided into categories based on the properties of the learning method: model-based versus model-free, on-policy versus off-policy strategies and value-based versus policy-based techniques. In model-based RL, algorithms create a model of the environment to simulate and predict the outcomes of actions. This allows the agent to plan by forecasting the states. This is possible when the environment is simple enough to be modelled. On the other hand, in model-free RL, algorithms directly learn a policy or value function from their interactions with the environment without explicitly modelling it. Model-free approaches are often chosen for environments where modelling is challenging or when an accurate model is not available. Within model-free RL, there are on-policy strategies, which learn about the policy that is currently used to make decisions, and off-policy strategies, which explore a different policy while aiming to learn an optimal one [7].

Another important distinction lies between value-based (also designated as critic-only) and policy-based (also designated as actor-only) learning approaches. On the one hand, value-based methods, like Temporal Difference Learning (TD), Q-Learning and SARSA Learning, focus on estimating the value of states or state-action pairs. These algorithms derive policies implicitly from these values. Temporal Difference learning is the update rule the other two are built on, and it applies in both settings: SARSA is on-policy, because it evaluates the policy it is following, while Q-Learning is off-policy, because it evaluates the greedy policy regardless of the one it follows. Deriving the policy by taking the highest-valued action requires comparing every action, which restricts these methods to discrete action sets. On the other hand, policy-based methods, such as REINFORCE, optimise the policy directly and therefore extend to continuous actions, where the policy returns the action itself rather than a value for each candidate. Actor-Critic learning combines the two. It holds a policy function (the actor) and a value function (the critic), so the policy is improved directly while the critic supplies the value estimates that make those updates stable [7]. Refer to Algorithm 2 for a detailed implementation.

Standard results for finite Markov Decision Processes guarantee that an optimal policy exists and can be taken to be deterministic and stationary, but those guarantees do not extend to complex, non-stationary environments such as financial markets [7]. A price series is not Markovian in its raw form, because the information a trader uses is spread across the recent history rather than contained in the

Algorithm 2 Actor-Critic [7].

Require: $\pi(a \mid s; \theta)$: a differentiable policy, $Q(s, a; w)$: a differentiable Q -function, N : number of iterations and learning rates $\beta^\theta > 0$ and $\beta^w > 0$
Initialise policy parameter $\theta^{(0)}$ and Q -function parameter $w^{(0)}$
Initialise state s
for $n = 0, 1, \dots, N - 1$ **do**
 Sample $a \sim \pi(\cdot \mid s; \theta^{(n)})$
 Take action a , observe state s' and reward r
 Sample $a' \sim \pi(\cdot \mid s'; \theta^{(n)})$
 Compute the TD error: $\delta \leftarrow r + \gamma Q(s', a'; w^{(n)}) - Q(s, a; w^{(n)})$
 Update w : $w^{(n+1)} \leftarrow w^{(n)} + \beta^w \delta \nabla_w Q(s, a; w^{(n)})$
 Update θ : $\theta^{(n+1)} \leftarrow \theta^{(n)} + \beta^\theta \delta \nabla_\theta \ln \pi(a \mid s; \theta^{(n)})$
 $s \leftarrow s'$
end for

latest quote. RL applied to these markets therefore works with an approximation, in which technical indicators and engineered features are used to fold enough of that history into the state for the Markov property to hold approximately. Actor-Critic methods matter here because they are the point at which reinforcement learning and neural networks meet, which is the subject of the next section.

2.4 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) combines the decision making of reinforcement learning with the function approximation of neural networks. The value functions and policies of Section 2.3 are defined over states, and in a tabular setting each state is stored separately. That is not possible when the state is a vector of continuous market variables, because the number of distinct states is unbounded. Replacing the table with a neural network removes that limit and allows the agent to generalise from states it has seen to states it has not.

2.4.1 Fundamentals and History

DRL emerged once it became clear that neural networks could act as the function approximators reinforcement learning needed in order to handle high-dimensional input. The result that established the field was the Deep Q-Network, which learned to play a range of Atari 2600 games directly from raw pixels and reached human-level scores on many of them [8, 33]. It showed that a single architecture could learn control policies from unprocessed, high-dimensional input, without features designed for each individual task. That result is what prompted interest in applying the same methods to financial markets, where the input is also high-dimensional and the mapping from observation to decision is also unknown. The difference is that an Atari game is stationary and fully observable, while a market is neither, and this is the reason results transfer far less directly than the analogy suggests.

2.4.2 Deep Value-Based RL Algorithms: Deep Q-Networks

Deep value-based methods use a neural network to approximate the Q -function, which allows them to operate over state spaces too large to enumerate. Neural Fitted Q Iteration was an early technique of this kind, fitting a network to the Q -values of a stored set of transitions and refitting it as more transitions were collected [34]. The approach became established with the Deep Q-Network (DQN) algorithm [8], which added two mechanisms that made training stable enough to be practical. The first is a separate target network, a delayed copy of the value network used to compute the learning targets, which prevents the target from moving at the same rate as the estimate. The second is experience replay, a buffer of past transitions from which mini-batches are drawn at random, which breaks the temporal correlation between consecutive samples. Both mechanisms reappear in the algorithm used in this work.

Following DQN, the Double Deep Q-Network (DDQN) addressed the tendency of DQN to overestimate action values [35]. It does so by separating action selection from action evaluation, using the online network to choose the action and the target network to score it. Deep value-based methods are not used in this work, because selecting an action by comparing the values of all available actions requires the action set to be discrete.

2.4.3 Deep Policy-Based RL Algorithms: Deep Policy Gradients

Deep policy-based algorithms optimise the policy directly instead of deriving it from action values, which is what allows them to operate on continuous actions. In trading this matters because a position has a size as well as a direction, and a method restricted to a discrete action set has to decide that size outside the learned policy. The Deep Deterministic Policy Gradient (DDPG) algorithm is a model-free, off-policy, actor-critic method for continuous action spaces [3]. It maintains four networks: an actor $\mu(s|\theta^\mu)$ that maps a state to a single action, a critic $Q(s, a|\theta^Q)$ that estimates the value of a state-action pair, and delayed copies of both, the target actor μ' and the target critic Q' . Transitions are stored in a replay buffer and mini-batches of N transitions are sampled from it at random. The critic is trained the same way as in DQN, by minimising the squared error between its estimate and a target built from the observed reward and the value of the next state [3]:

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})) \Big| \theta^{Q'} \quad (21)$$

$$L = \frac{1}{N} \sum_i \left(y_i - Q(s_i, a_i|\theta^Q) \right)^2 \quad (22)$$

The actor is updated by the deterministic policy gradient, which follows the direction in which the critic's estimated value increases. Because the action is continuous, the gradient of the value with respect to the action is defined, and the chain rule carries it back to the actor's parameters [3]:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q) \Big|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu) \Big|_{s_i} \quad (23)$$

The target networks are not copied outright. They are moved a small fraction $\tau \ll 1$ towards the trained networks after every update, which keeps the learning targets almost stationary at the cost of slowing how fast they track improvements [3]:

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta' \quad (24)$$

Since the policy is deterministic, it produces no exploration of its own, and the problem raised in Section 2.3 returns here. DDPG solves it by adding a noise process $\mathcal{N}$ to the action during training, so that the action actually executed is $\mu(s|\theta^\mu) + \mathcal{N}$. The original work uses an Ornstein-Uhlenbeck process, which produces noise correlated in time rather than independent at each step, on the grounds that a physical system with momentum explores better under a disturbance that persists for a while than under one that changes sign at every step [3]. Algorithm 3 sets out the complete procedure.

DDPG is powerful but not dependable. It carries no convergence guarantee outside restrictive conditions, and it is known to be sensitive to its hyperparameters and to the random seed, so two runs of the same configuration can end at different policies. A market makes this worse, since it is non-stationary and only partially observable, and the guarantees that apply to finite Markov Decision Processes do not hold. These are properties of the method rather than obstacles to be removed, and Section 5 treats them as such by reporting how results vary rather than reporting a single run.

The four topics covered here are the parts this work combines: a market to trade in, networks to approximate the functions involved, a reinforcement learning formulation of the decision problem, and an algorithm that solves it for continuous actions. Combining them is not new, and a considerable body of published work has already attempted it.

Those studies differ in how they define the state, the action and the reward, and in whether the agent is charged for trading at all. Those choices decide whether a reported result would hold on a live

Algorithm 3 Deep Deterministic Policy Gradient [3].

Require: an actor $\mu(s|\theta^\mu)$, a critic $Q(s,a|\theta^Q)$, a discount factor γ , a soft update rate $\tau \ll 1$ and a minibatch size N

Randomly initialise the critic parameters θ^Q and the actor parameters θ^μ

Initialise the target parameters $\theta^{Q'} \leftarrow \theta^Q$ and $\theta^{\mu'} \leftarrow \theta^\mu$, and the replay buffer $\mathcal{B}$

for episode = 1, M **do**

 Initialise a random process $\mathcal{N}$ for action exploration

 Receive the initial observation state s_1

for $t = 1, T$ **do**

 Select the action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ from the current policy and the exploration noise

 Execute the action a_t and observe the reward r_t and the new state s_{t+1}

 Store the transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{B}$

 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from $\mathcal{B}$

 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

 Update the critic by minimising the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

 Update the actor policy using the sampled policy gradient:

$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$

 Update the target networks:

$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$

$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$

end for

end for

account, which makes them the right basis for comparing the literature and for placing this work within it.

3 Related Work

This section reviews the published work on deep reinforcement learning applied to financial trading. The review was carried out in two stages. Surveys were read first, to establish which problems the field treats as open and which papers are worth reading in full. Individual articles were then read and their design decisions recorded in a common format, so that studies using different markets and different algorithms could be compared on the same terms.

Eighteen articles were examined in that way. Their basic features are collected in Table 9 and the design decisions inside each agent in Table 10, and the discussion below refers to both throughout.

3.1 Survey Research: Identifying the State-of-the-Art

The review began with surveys, which give the shape of a field faster than individual papers do and point back to the work that established each result.

Hambly et al. [10] provided the theoretical basis. The survey sets out reinforcement learning and neural networks separately, then covers how the two combine into deep reinforcement learning and how the result has been applied in finance. It gives the formulations and pseudocode for critic-only, actor-only and actor-critic algorithms together with the papers that introduced each of them, and it is the reference used while writing Section 2.

Fischer [9] served the opposite purpose. It is concerned with practice rather than theory, and it reviews implementations rather than derivations. Its value here was the set of dimensions along which it compares studies: the dataset, the network beneath the reinforcement learning agent, the state space, the action space, the reward function, the techniques used against the bias-variance and exploration-exploitation trade-offs, and the training procedure. Those dimensions were adopted as the columns of Table 9 and Table 10, so that the articles reviewed below could be compared consistently rather than described one at a time.

Three further surveys were consulted [11, 13, 36]. They agree on the broad picture: the field is dominated by equity markets, daily data and discrete trading decisions, and reported results are hard to compare because the trading environment differs from paper to paper.

3.2 Article Research: Summarising Different Implementations

Eighteen articles with working implementations were then read in full. Their features were recorded in two tables covering the same articles: Table 9 holds the market, the data and the algorithm used, and Table 10 holds the design decisions inside each agent. The articles are grouped below by the market they trade.

3.2.1 Forex Market Implementations

Three of the eighteen articles trade currencies, and all three restrict the agent to a discrete choice between buying, holding and selling.

Carapuço et al. [14] implemented a Deep Q-Network over a fully connected network on raw tick data from 2010 to 2017. The agent was allowed to act only once a fixed number of ticks had passed, which produced a decision roughly every two hours. It outperformed its benchmark by a wide margin, and it is the only study reviewed here that works from tick data rather than from bars. Huang [15] used a Deep Recurrent Q-Network, which places a long short-term memory network beneath the Q-function, on 15-minute bars from 2012 to 2017. The model outperformed its benchmark with explicit and implicit trading costs charged, which puts it among the few results in the set that survive a cost model. Rundo [16] applied the same class of model to one-minute bars from 2004 to 2018 and also reported outperformance, but with an accuracy high enough to suggest look-ahead bias and with a large maximum drawdown. Read together, the two results suggest that adding recurrence does not by itself make very short timeframes tractable.

Reference	Market	Dataset	Timeframe	Period	NN	RL	DRL	Key Findings
[14]	Forex	Raw Ticks	≈ 2 Hours	2010 - 2017	FNN	Critic-Only	DQN	Significantly outperformed the benchmark.
[15]	Forex	OHLCV	15 Minutes	2012 - 2017	RNN (LSTM)	Critic-Only	DRQN	Significantly outperformed the benchmark even with explicit and implicit costs.
[16]	Forex	OHLCV	Minute	2004 - 2018	RNN (LSTM)	Critic-Only	DRQN	Outperformed the benchmark, but had abnormally high accuracy and maximum drawdown.
[37]	Stocks	OHLCV, News	Daily	≈ 5 Months	RNN (LSTM)	Actor-Critic	DDPG	Outperformed the benchmark and News are important features.
[38]	Stocks	OHLCV	Monthly	2005 - 2019	FNN	Actor-Critic	DDPG	Outperformed the benchmark even with explicit and implicit costs.
[39]	Stocks	OHLCV	Daily	2016 - 2020	FNN	Actor-Critic	DDPG, A2C, PPO	The ensemble underperformed the benchmark and had unstable returns.
[40]	Stocks, Futures	OHLCV	Minute	2010 - 2020	FNN (SDAE), RNN (LSTM)	Critic-Only, Actor-Critic	DQN, A3C	Both significantly outperformed the benchmark and A3C had superior results and convergence.
[41]	Stocks, Cryptos	OHLCV	Daily	2010 - 2021	FNN	Actor-Critic	DDPG (TD3)	Outperformed the Cryptos benchmark but underperformed the Stocks benchmark.
[42]	Stocks	OHLCV	Daily	1992 - 2020	FNN	Critic-Only, Actor-Critic	DQN, DDPG, PPO, A2C	All outperformed the benchmark, PPO and DQN had superior results and more consistent, DDPG and A2C performance was below expectation.
[43]	Stocks	OHLCV	Daily	2006 - 2022	FNN	Critic-Only	DQN	Outperformed the benchmark with good generalization power in out-of-sample assets.
[44]	Stocks	OHLCV	Daily	2012 - 2019	FNN	Critic-Only	DQN	Outperformed the benchmark with consistent results in stable and volatile markets.
[45]	Stocks	OHLCV	Daily	2008 - 2018	RNN (GRU)	Critic-Only, Actor-Critic	GDQN, GDPG	Both outperformed the benchmark with consistent results in stable and volatile markets. GDPG had superior results.
[46]	Stocks	OHLCV	Daily	2009 - 2018	FNN	Actor-Critic	DDPG	Outperformed the benchmark.
[47]	Stocks	OHLCV	Daily	2009 - 2020	FNN	Actor-Critic	DDPG, A2C, PPO	The ensemble significantly outperformed the benchmark with explicit costs, then PPO had the best results, followed by A2C and finally DDPG.
[48]	Stocks	OHLCV	Daily	2009 - 2021	FNN	Actor-Critic	DDPG, A2C	Both outperformed the benchmark. DDPG had superior results, convergence and stability due to experience replay and noise.
[49]	ETFs	OHLCV	Daily	2000 - 2018	FNN, RNN (LSTM)	Critic-Only	DQN, DRQN	Both outperformed the benchmark and DRQN had superior results.
[50]	ETFs	OHLCV	Daily	2002 - 2020	RNN (LSTM)	Actor-Critic	DDPG	Outperformed the benchmark using variable position sizing and explicit and implicit costs.
[51]	Futures	OHLCV	Daily	2005 - 2019	RNN (LSTM)	Actor-Critic	A2C	Outperformed the benchmark with explicit and implicit costs.

Table 9: Summary of the basic features of related work articles.

Reference	State Space	Action Space	Reward Function	Bias/Variance	Exploration/Exploitation	Training Techniques
[14]	Ask, Bid, Volume, Technical Analysis	Discrete (1, 0, -1)	Total Return	Tweak Network, Early Stopping	Policy ϵ -greedy	Cross-Validation
[15]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Total Return	Tweak Replay Memory, Tweak RNN Sequence	Policy ϵ -greedy	Online Learning
[16]	OHLCV	Discrete (1, 0, -1)	Total Return	Tweak Network	-	Train-Validate-Test Split
[37]	OHLCV, Sentiment and Technical Analysis	Discrete (1, 0, -1)	Total Return	Tweak Network	Policy ϵ -greedy, Shuffling	Cross-Validation
[38]	Log Returns, Volume	Discrete (1, 0, -1)	Total Return	-	-	Pre-Train with GA
[39]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Sharpe Ratio	Ensemble Selection	Ensemble Selection	Cross-Validation
[40]	OHLCV, Metrics, Technical Analysis	Discrete (-n, ..., 0, ..., n)	Total Return	Denosing	Gaussian Noise	Adaptive Learning Rate
[41]	OHLCV	Continuous [-1, 1]	Log-Return	Normalisation, Regularisation	Policy ϵ -greedy Gaussian Noise	Train-Validate-Test Split
[42]	OHLCV	Discrete (1, 0, -1)	Sharpe Ratio	Wavelet Transform	-	Train-Validate-Test Split
[43]	OHLCV, Macroeconomic Metrics	Discrete (1, 0, -1)	Total Return, (1, 0, -1)	Normalisation, Regularisation	Policy ϵ -greedy (Adaptive)	Adaptive Synchronization
[44]	OHLCV	Discrete (1, -1)	Total Return	Normalisation, Regularisation	Duplicate Environment (Inverse Positioning)	Adaptive Learning Rate, Huber Loss, Gradient Clip
[45]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Total Return	Normalisation, Regularisation	Policy ϵ -greedy Gaussian Noise	Train-Test Split
[46]	OHLCV	Discrete (1, 0, -1)	Total Return	Tweak Network	Gaussian Noise	Train-Validate-Test Split
[47]	OHLCV, Technical Analysis	Discrete (-n, ..., 0, ..., n)	Total Return (Turbulence Index)	Ensemble Selection	Ensemble Selection	Train-Validate-Test Split
[48]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Total Return	Normalisation	Gaussian Noise	Train-Test Split
[49]	OHLCV	Discrete (1, 0, -1)	Total Return	Tweak Learning Rate	-	-
[50]	OHLCV	Continuous (Position Sizing)	Sharpe Ratio	-	Gaussian Noise	Train-Validate-Test Split
[51]	OHLCV, Technical Analysis	Continuous [-1, 1]	Total Return (with Volatility)	Normalisation	-	Cross-Validation

Table 10: Summary of the advanced features of related work articles.

3.2.2 Stock Market Implementations

Equity markets account for most of the work reviewed. Among the critic-only studies, Sim and Kirk [43] trained a Deep Q-Network on daily bars from 2006 to 2022 and tested it on assets it had not seen, reporting that the policy transferred. Théate and Ernst [44] used the same algorithm on daily bars from 2012 to 2019 and reported consistent results in calm and volatile periods alike, obtaining exploration from an inverted duplicate of the environment rather than from a noise process. Li et al. [40] compared a Deep Q-Network over a denoising autoencoder against Asynchronous Advantage Actor Critic over a long short-term memory network, on minute bars of stocks and futures from 2010 to 2020; both beat the benchmark, and the asynchronous actor-critic converged better, which the authors attribute to its parallel training.

The actor-critic studies are closer to the design used here. Xiong et al. [46] applied DDPG to daily bars from 2009 to 2018 and outperformed the benchmark. Yousefi [48] compared DDPG against Advantage Actor Critic on daily bars from 2009 to 2021, finding DDPG the more stable of the two and crediting its replay buffer and its noise process. Gran et al. [38] ran DDPG on monthly bars from 2005 to 2019 and kept its advantage once explicit and implicit costs were charged. Azhikodan et al. [37] added sentiment drawn from news to a state space that already held price data and technical indicators, and reported that the addition improved results, although the evaluation covers roughly five months and the exact period is not stated. Majidi et al. [41] used the Twin Delayed variant of DDPG on daily bars from 2010 to 2021 across stocks and cryptocurrencies, with a continuous action in $[-1, 1]$; it beat the cryptocurrency benchmark but not the equity one.

Sagiraju and Mogalla [42] deserve separate mention. They compared four algorithms on daily bars from 1992 to 2020 and, while all four beat the benchmark, Proximal Policy Optimisation and the Deep Q-Network proved the more consistent, with DDPG and Advantage Actor Critic falling below expectation. It is the one study in the set that reports DDPG performing worse than its alternatives, which is a useful counterweight to the studies above.

Two further studies combine algorithms rather than choosing between them. Yang et al. [47] ran DDPG, Advantage Actor Critic and Proximal Policy Optimisation together on daily bars from 2009 to 2020, switching between them on a turbulence measure, and the ensemble beat the benchmark with explicit costs charged. Kong and So [39] combined the same three algorithms on daily bars from 2016 to 2020 and obtained unstable returns and underperformance. The two results together indicate that combining agents is not by itself a source of robustness. Wu et al. [45] sit between the groups, comparing a gated recurrent Q-network against a gated recurrent policy-gradient agent on daily bars from 2008 to 2018, with the policy-gradient variant performing better.

3.2.3 Other Market Implementations

Three articles trade instruments outside equities and currencies. Chen and Gao [49] traded exchange-traded funds on daily bars from 2000 to 2018, comparing a Deep Q-Network over a fully connected network against the recurrent variant, with the recurrent model performing better. Zhang et al. [51] applied Advantage Actor Critic over a long short-term memory network to futures on daily bars from 2005 to 2019, keeping its advantage after costs. Jia et al. [50] also traded exchange-traded funds, on daily bars from 2002 to 2020, using DDPG over a long short-term memory network. They tested two variants, one holding a fixed position size and one sizing the position continuously, and the variable-size version performed better with explicit and implicit costs charged. It is the closest study in the set to the design used here, being the only one that uses a continuous action to size a position and charges for trading at the same time.

3.2.4 Comparative Findings

Reading the two tables across their columns rather than down their rows shows where the field has settled.

The state space is normally price data with technical indicators derived from it. Studies reporting strong results almost always include indicators rather than price alone [14, 15, 47]. That is consistent with

the argument in Section 2.3 that a raw price series is not Markovian, and that indicators fold enough of the recent history into the state for the property to hold approximately.

The action space is discrete in 15 of the 18 studies, usually the three choices of buy, hold and sell [16, 37, 46]. Three studies use a continuous action [41, 50, 51], and only one of those uses it to size a position rather than to express a direction [50]. A discrete action set is easier to learn, but it moves position sizing outside the policy, where it has to be set by a rule the agent cannot adapt. The reward is the total return in 14 of the 18 studies. Three use the Sharpe ratio [39, 42, 50] and one uses the log return [41]. A risk-adjusted reward is more expensive to compute during training, since it needs a window of past returns rather than the most recent one, and the studies that use it do not report a consistent advantage from doing so.

Against the bias-variance trade-off the common techniques are feature engineering, normalisation, regularisation and adjustment of the network itself, with denoising [40] and normalisation combined with regularisation [44] as representative examples. For exploration, the ϵ -greedy policy is standard where the action set is discrete and additive Gaussian noise where it is continuous [41, 45].

Trading costs are recorded inconsistently. Five of the 18 studies state that their headline result survives explicit and implicit costs [15, 38, 47, 50, 51], and the remainder report returns without recording what was deducted, which leaves the reader to guess whether the figure is gross or net. Training is normally a train-validation-test split, sometimes with cross-validation. One study trains online, continuing to learn as new data arrives [15], which is the closest any of them comes to the conditions of live deployment.

The data is long but coarse. Twelve of the 18 studies cover ten years or more, with a median span of eleven years, so a long history is the norm rather than the exception. Resolution is where they are alike in the other direction: 17 of the 18 work on bars, 13 of those on daily bars, and only one uses quote-level data [14]. The distinction matters for cost accounting, since a bar reports four prices and conceals the order in which the price visited them.

3.3 Research Gap: Positioning This Work

The eighteen studies agree that a deep reinforcement learning agent can learn a profitable trading policy. Where they differ, or stay silent, is on the conditions under which that holds. The findings above describe what the field does; read against one another they also show what it has not yet done. Table 11 sets out the comparison.

Dimension	Surveyed literature (18 articles)	This work
Market	Stocks 12, Forex 3, ETFs 2, Futures 1	Forex, the seven major pairs
Data resolution	Bars in 17 of 18, daily in 13; one study works from quotes	Quote-level tick data, with hourly and daily decisions
Data span	Ten years or more in 12 of 18, median eleven years	Eleven years, 2015 to 2026
Action space	Discrete 15, continuous 3; one sizes the position	Continuous, setting direction and size together
Reward function	Total return 14, Sharpe ratio 3, log return 1	Log return, with selection on the Calmar ratio and a risk-adjusted composite
Trading costs	Charged in 5 of 18	Quoted spread at the moment of the trade plus the commission of a swap-free raw-spread account
Instruments per design	One agent per instrument, or one agent over a basket	Seven pairs trained separately under one design, then combined
Reported robustness	Neither the candidate count nor the sensitivity to starting conditions is stated	Five starting balances per pair, with the candidate count stated

Table 11: Position of this work against the eighteen surveyed articles.

The first gap appears only when two of those columns are read together. The three studies that trade

currencies [14–16] all restrict the agent to a discrete choice, and the three that give the agent a continuous action [41, 50, 51] all trade something else. Among the work reviewed here the two properties never occur together, so there is no direct comparison available for a currency agent that sets its own position size. That is the combination this work uses, and the absence is the reason it cannot be measured against an existing result.

The second gap follows from the cost figures. A strategy evaluated without charges is not evaluated against the market it claims to trade, and the omission matters more in Forex than the reported returns suggest, because the spread is not fixed. It widens when liquidity falls and around economic releases, which are the moments at which a model trained on an average spread would expect to act. The third gap is resolution rather than length of history. A long sample is already the norm in this literature, so it is not by itself distinctive; working inside the bar is. A bar reports four prices and conceals the order in which the price visited them, and that order decides which spread was paid and whether a stop was reached before a target.

The fourth gap is one of evidence rather than of design. Each study reports the performance of a model it selected, and none of those reviewed here states how many candidates that model was selected from, or how the reported figure moves when the evaluation is repeated from a different starting condition. Deep reinforcement learning is sensitive to initialisation, so a single figure does not separate the contribution of the method from the contribution of the run, and a model chosen as the best of many describes the sample it was chosen from as much as the market it traded [12, 17]. The last gap concerns how results are combined. The studies reviewed report either a single agent trading one instrument, or a single agent trading a basket. None of them trains one design separately on each of several instruments and then measures what those agents do when they are run together, which is how a trader holding several currency pairs would deploy them. The combination is not implied by the individual results, because it depends on how the equity curves move relative to one another.

This work is positioned in those gaps. One design is applied unchanged to all seven major currency pairs, with a continuous action that sets the direction and the size of the position together. The evaluation runs on quote-level data across eleven years and charges the spread quoted at the moment of each trade together with the commission of a swap-free raw-spread retail account, which is an account a retail trader can open rather than a modelling convenience. Each pair is reported from five different starting balances rather than from a single backtest, and the number of candidates the selected model was drawn from is stated. The seven agents are then combined into an equal-weight portfolio, so the work reports both the seven individual results and the single result a trader running all of them would have seen. Section 5 gives both.

One point of similarity is worth stating alongside the differences. The reward is the log return, which is the choice made by one of the surveyed studies [41] and is not a point of difference. What differs is that the reward the agent optimises and the criteria the final model is selected on are deliberately not the same measure. Candidate runs are scored during training by the Calmar ratio, and the model finally published for each pair is the one ranked highest by a composite of percentile ranks within its own pool of candidates, built from the annualised return, the Sharpe, Sortino, Calmar and Sterling ratios defined in Section 2.1.5, the maximum and downside risk, and measures of how the policy behaves. A policy that earns its return by taking more risk therefore does not win on return alone. Section 4 sets out both.

The studies reviewed here answer the question of whether a deep reinforcement learning agent can learn a profitable trading policy. They say much less about the conditions under which that answer holds, because the environment each agent was trained in is usually described in a sentence or two and is rarely reproducible from the description given.

That environment is the part this work had to build before the question could be asked. An agent trained against a simplified market learns to exploit the simplification, so the worth of any result depends on how the environment charges for trading, advances time and fills orders. Section 4 describes that environment and the agent that runs inside it.

4 Implementation

Section 3 set out what the reviewed literature leaves open. This section describes what was built in response: the environment the agent trades in, the agent itself, and the procedure by which it was trained and a single model chosen for each currency pair.

The order is deliberate. An agent cannot be described apart from the environment that supplies its observations and charges it for its actions, and no number produced by it means anything unless that environment is faithful. The framework therefore comes first, the agent second, and the experimental procedure third.

4.1 Trading Framework: The Environment the Agent Trades In

No framework was available that could both train an agent offline on quote-level history and run that same agent on a live account without rewriting it. Building one took a substantial part of the effort behind this work, and the design choices in it determine what the results in Section 5 are worth. Three properties were required. The tested system and the deployed system had to be the same code, so that a result obtained offline says something about live behaviour. The data had to be fine enough to reconstruct the price path inside a bar, because that path decides what a position actually paid. And the costs had to be the costs of a real account rather than a modelling convenience.

4.1.1 System Architecture: Three Execution Contexts

The framework is split across two languages by necessity. Strategy logic, the learning models and the market abstractions are written in Python, where the numerical and machine learning libraries are. Access to the broker is written in C#, because cTrader [18] executes algorithmic strategies as cBots on the .NET runtime and offers no other route to its order book. The two halves communicate through named memory-mapped files with paired auto-reset event handles, one pair for each direction, operating as a single-slot request and response exchange: the platform writes an update and signals, Python reads it, writes an action and signals back, and the platform proceeds. Two message families cross that boundary, updates travelling from the platform to Python and actions travelling the other way. Figure 4 shows the arrangement. What matters is not the transport but the consequence of it: the strategy object is unaware of where its updates originate. The same class, holding the same weights, runs in three contexts.

In the first context the platform is absent. Python replays stored quotes directly from the market database, which is the fastest of the three and the one used for training and for every result reported in this work. In the second the strategy runs inside cTrader’s own backtesting tab against the platform’s historical data, which serves as an independent check on the offline engine. In the third the same strategy runs on a live account, receiving real quotes and sending real orders. The value of this arrangement is that the three contexts differ only in the source of the updates. A strategy that behaves one way in a backtest and another when deployed is usually the product of two separate implementations that drifted apart; here there is one implementation, and the question does not arise.

4.1.2 Market Data: Collection, Storage and Resolution

The historical data was collected through the same connector, requested from cTrader by the C# component and written to a local PostgreSQL database by the Python one. Two record types are stored. Ticks hold the bid and ask quoted at an instant, together with the conversion rates needed to express a profit or loss in the account currency at that instant. Bars hold open, high, low and close as references to the ticks that produced them, so a bar is a view over the tick tape rather than a separate copy of the prices. Table 12 gives the inventory for the seven pairs. The tick tape is the primary record and the bar series are derived from it, which is what allows the engine to reconstruct the path a price took inside a bar rather than interpolating between its endpoints.

Two properties of this data need stating, because both would otherwise look like defects.

Bars are stamped at the close of the broker’s trading session rather than at midnight, at 22:00 UTC or 21:00 under daylight saving. A daily series therefore carries five bars in a normal week, stamped Sunday

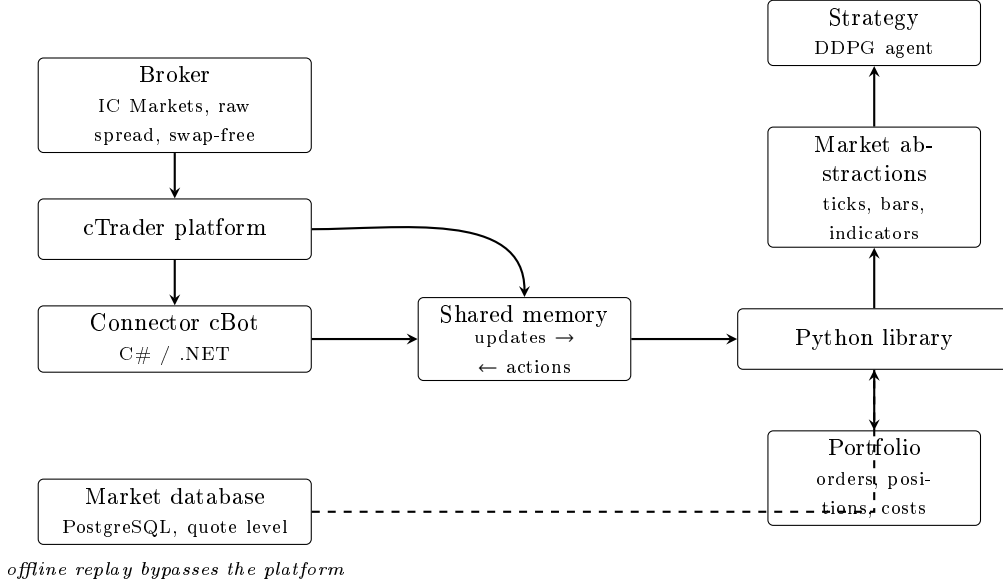


Figure 4: Framework architecture. The strategy receives updates and returns actions without knowing whether they originate from a live account, from the platform's own backtester, or from the stored quote database.

Pair	Ticks	M1 bars	H1 bars	D1 bars	MN1 bars
AUD/USD	227,914,614	4,643,990	78,320	3,346	150
EUR/USD	242,847,954	4,636,138	78,145	3,339	150
GBP/USD	281,490,557	4,648,216	78,311	3,339	150
NZD/USD	203,788,307	4,631,614	78,282	3,339	150
USD/CAD	228,356,007	4,635,179	78,319	3,347	150
USD/CHF	194,186,979	4,621,503	78,307	3,338	150
USD/JPY	284,430,458	4,638,775	78,151	3,338	150
Total	1,663,014,876	32,455,415	547,835	23,386	1,050

Table 12: Market data collected from cTrader and stored locally, spanning January 2014 to August 2026. The tick tape occupies 295 GB and is the primary record; the bar series reference the ticks that produced them.

through Thursday, and the bar stamped Thursday is Friday's session. An audit against a calendar-day reference reports every Friday as missing, which is a convention rather than a loss. The period from 11 to 25 January 2016 is absent from all seven pairs, in ticks and in bars alike. The gap is broker-side and was confirmed unrepairable by a re-download that visited those days and returned nothing. It is uniform across the seven pairs, so it does not bias any comparison between them, and it falls outside the evaluation window used in Section 5.

4.1.3 Cost Model: Spread, Commission and Account Type

The execution engine advances on ticks rather than on bar closes. A position opened during a bar is filled at a quote that existed within that bar, and a stop or target attached to it is tested against the quotes in the order they arrived. This matters for the reason given in Section 3: a bar reports four prices and conceals the sequence in which the price visited them, and that sequence decides whether a stop was reached before a target and which spread was crossed.

The account modelled is a swap-free raw-spread retail account, and each of the three charges described in Section 2.1.5 is applied at the rate such an account pays. A buy is filled at the ask and a sell at the

bid, both read from the tick tape at the instant of the fill, so the spread is charged by construction rather than deducted afterwards. The commission is charged on traded volume at 3.5 points per side, which for a standard lot of 100,000 units is 3.5 units of the quote currency per side, or 7 per round turn. The swap is zero, which on a swap-free account is not an approximation but the rate the account actually charges. Table 13 lists the full configuration.

Setting	Value	Meaning
Account type	Raw spread, swap-free	An account a retail client can open, quoting the interbank spread and charging no overnight financing
Spread	Accurate	The bid-ask difference recorded at the moment of the trade, read from the tick tape rather than assumed
Commission	3.5 points per side	Charged on traded volume; equivalently 3.5 units of the quote currency per standard lot per side, or 7 per round turn
Swap	0 points	The rate a swap-free account charges; not an approximation
Account currency	EUR	Profit and loss converted at the rate recorded on the tick that produced it
Initial balance	10,000	Reported across five starting balances, as described in Section 5
Leverage	30	The retail limit applicable to major currency pairs
Position mode	Netting	One net position per instrument, rather than independent hedged positions

Table 13: Cost and account configuration applied to every evaluation reported in this work.

The decision to read the spread rather than assume it is worth defending with the data, because the assumption is common and the error it introduces is not small. Figure 5 plots the monthly mean spread of each pair across the evaluation window. For EUR/USD, the most liquid pair in the market, the monthly mean ranges from 0.77 to 13.00 points, a factor of seventeen between the quietest month and the busiest. USD/CHF reaches 37.82 points in January 2015, when the Swiss National Bank abandoned its floor against the Euro. Across the seven pairs the ratio between the widest and narrowest month is never below five.

The shape of that variation matters more than its size. Spreads widen when liquidity falls and around the events that move prices, which are the moments at which a model trained on an average spread would most expect to act. A strategy charged a constant spread is therefore not merely charged the wrong amount; it is charged the wrong amount in a way that is correlated with its own activity, and the error runs in the direction that flatters it.

The engine was validated against cTrader’s own backtester rather than against itself. Across six reference configurations, covering two currency pairs and three windows, its trade, position, order and deal records reproduce the platform’s output byte for byte, and the conversion of profit and loss into the account currency was checked independently against the platform’s own figures; the only divergence is sub-pip and follows from the resolution of the stored quotes rather than from the accounting. The engine therefore reproduces the platform it was built against, and then goes beyond it. The same strategy is evaluated offline directly on the tick tape, at a speed the platform’s own backtester cannot reach and over windows it will not span, under a cost model that can be set to any account the broker offers rather than to the one the terminal happens to be connected to.

Two consequences follow, and both are stated here so that Section 5 can be read correctly. The returns reported in this work are net of the spread actually quoted and of a real commission schedule, so they are lower than the same policy would show on bar closes with a fixed or absent spread, and they are not directly comparable with results obtained that way. And because the account is swap-free, no financing term is estimated or omitted; a strategy that holds positions across many nights is charged for exactly what such an account would charge it.

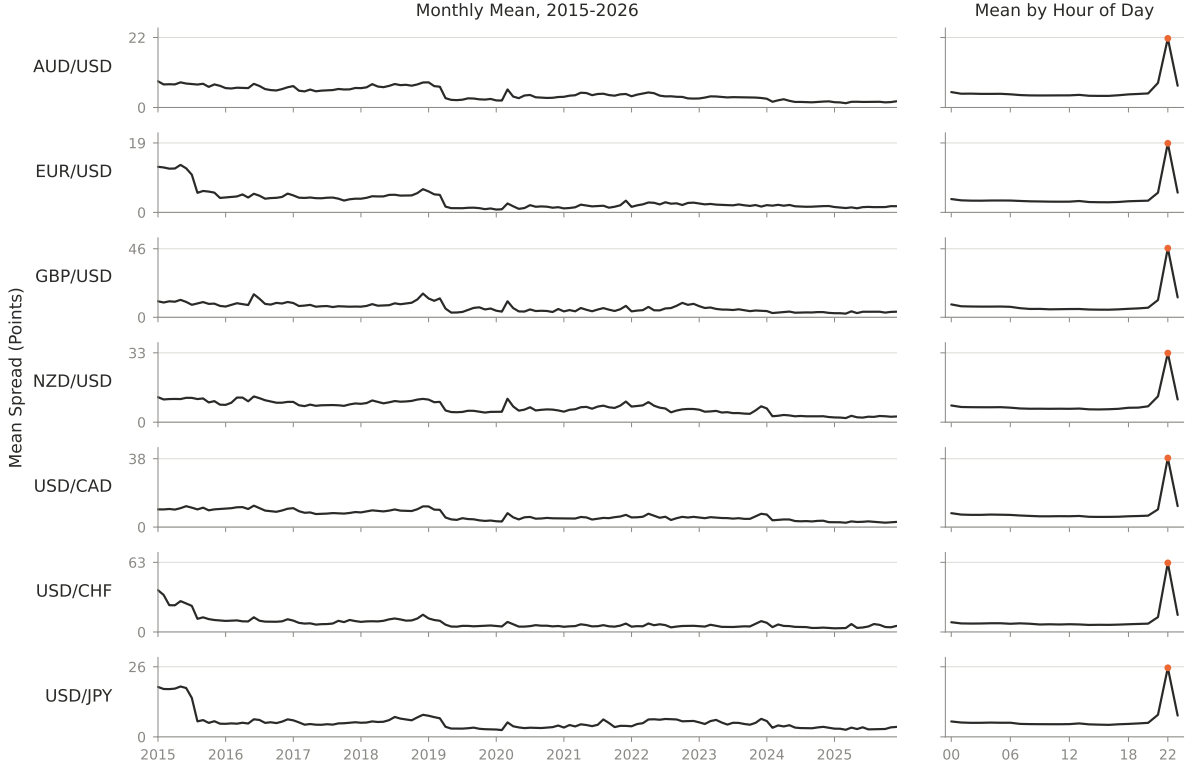


Figure 5: Monthly mean spread in points for the seven major pairs, 2015 to 2026, computed from the full tick tape. EUR/USD is drawn in black. A model evaluated at a fixed spread is evaluated against a market that exists in no month of this sample.

4.2 Agent Design: Observation, Action and Reward

The environment of Section 4.1 supplies prices and charges for trades. Turning it into a reinforcement learning problem requires three further decisions: what the agent sees, what it is allowed to do, and what it is paid for. This subsection states those three and then the algorithm that learns from them. Each decision is reported with the reasoning behind it, because in every case the alternative was tried and behaved worse, and those failures are part of the result.

4.2.1 Observation Space: Indicators, Encoding and Normalisation

The agent observes a vector of 38 values at each hourly bar. Nothing in it is a raw price. A price level carries no information the agent can generalise from, because a policy learned at 1.05 has no reason to transfer to 1.20, and eleven years of history spans ranges that differ by more than that. Every feature is therefore a ratio, and the whole observation is dimensionless. Sixteen technical indicators, listed in Table 14, supply the market context. They divide into a fast bank describing the last hours to days and a slow bank describing regimes measured in months, so that the agent can distinguish a move within a trend from a change of trend.

The indicator values are encoded as follows, writing C for the bar close, σ_f and σ_s for the fast and slow realised volatilities, A for the average true range, and M_i for the i -th moving average with period N_i :

$$\frac{A}{C}, \quad \sigma_f, \quad \ln\left(\frac{\sigma_f}{\sigma_s}\right), \quad \text{ER} \quad (25)$$

Indicator	Family	Period	Role in the observation
ATR	Volatility	14	Volatility unit; divides every trend feature and sets the stop distance
RV (fast)	Volatility	16	Short-horizon realised volatility; divides the momentum features
RV (slow)	Volatility	480	Long-horizon realised volatility; its ratio to the fast reading gives the volatility regime
ER	Trend	120	Efficiency ratio; how directional recent movement has been, bounded in $[0, 1]$
ROC Fast	Momentum	24	One day
ROC Medium	Momentum	120	One week
ROC Slow	Momentum	480	One month
ROC Regime	Momentum	1,440	One quarter
ROC Epoch	Momentum	2,880	Two quarters
SMA Fast	Trend	24	One day
SMA Medium	Trend	120	One week
SMA Slow	Trend	480	One month
SMA Regime	Trend	1,440	One quarter
SMA Epoch	Trend	2,880	Two quarters
SMA Cycle	Trend	4,320	Three quarters
SMA Era	Trend	5,040	Roughly one year

Table 14: The sixteen technical indicators forming the market context of the observation. Periods are in hourly bars. Each moving average contributes two features, its distance from price and its own differential, both in units of the average true range; each momentum reading contributes one, scaled by the movement expected over its lookback.

$$\frac{\text{ROC}(N_j)}{\sigma_f \sqrt{N_j}}, \quad \frac{C - M_i}{A}, \quad \frac{M_i - M_i^{-1}}{A} \quad (26)$$

Each construction removes a different dependence. Dividing the average true range by the close expresses volatility as a fraction of price. The logarithm of the fast to slow volatility ratio is a volatility regime indicator that is zero when the two agree, positive when volatility is rising and negative when it is falling. Dividing a momentum reading by $\sigma_f \sqrt{N_j}$ scales it by the movement that would be expected over that lookback under a random walk, so that a value near one means a move of ordinary size regardless of the period. Measuring the distance from a moving average in units of the average true range states how far price sits from its trend in terms of the volatility currently being experienced, and the differential of the same quantity gives the direction that trend is moving. The remaining features are the calendar position encoded as four sine and cosine pairs, the current signed exposure as a fraction of the maximum the account could hold, and six values describing the last bar as log returns of its open, high, low and close against the previous close, divided by σ_f .

Features that are not bounded by construction are standardised by a causal exponentially weighted z-score with $\alpha = 1/200$. Writing x_t for a raw value and μ_t , s_t^2 for its running mean and variance:

$$z_t = \frac{x_t - \mu_{t-1}}{\sqrt{s_{t-1}^2 + \epsilon}} \quad (27)$$

$$\mu_t = \mu_{t-1} + \alpha \delta_t, \quad s_t^2 = (1 - \alpha) (s_{t-1}^2 + \alpha \delta_t^2), \quad \delta_t = x_t - \mu_{t-1} \quad (28)$$

The subscript $t - 1$ in Equation 27 is the point of the design. A value is standardised using statistics that end at the previous step, and only afterwards is it folded into those statistics. A z-score computed over the whole sample would leak the future into every observation and inflate every result derived from it. Features already bounded, such as the exposure fraction, the efficiency ratio and the calendar encodings, carry a flag that bypasses this layer so that their natural range is preserved.

4.2.2 Action Space: Position Sizing and Risk

The agent emits a single continuous value $a \in [-1, 1]$. Its sign is the direction of the position and its magnitude is the fraction of a reference size to hold, so direction and size are one decision rather than two. This is the property that the discrete action sets surveyed in Section 3 cannot express. The reference size is fixed-fractional. Writing B for the account balance, ρ for the risk fraction per position and k for the stop distance in multiples of the average true range, the reference volume V_{ref} is the volume that loses ρB if price moves $k \cdot A$ against it, and the target volume is that reference scaled by the action:

$$V_{target} = a \cdot V_{ref}, \quad V_{ref} = \frac{\rho B}{k A} \quad (29)$$

with $\rho = 1\%$ and $k = 1.5$ throughout. The agent therefore never chooses an absolute number of units; it chooses a fraction of a size that the risk rule has already set, and that size shrinks automatically as volatility rises.

One defect in this arithmetic was found during the campaign and is reported here because it affects one of the seven results. The conversion between the quote currency and the account currency is absent from the volume calculation, so the risk actually taken is ρ/p rather than ρ , where p is the price of the pair. For pairs quoted near unity the difference is immaterial, and the six pairs quoted against the US Dollar all take approximately the intended 1%. For USD/JPY, quoted near 120 at the start of the sample, the effective risk is 0.008%, small enough that the computed volume falls below the broker’s minimum and no position can open at all. The compensation applied was to scale ρ by the price, restoring the intended risk per position rather than adding leverage; Section 5 reports the level used for each pair. The underlying defect is a limitation of the framework rather than of the method, and it is recorded as future work.

4.2.3 Reward Function: Log Return, Scaling and Clipping

The reward at each step is the log return of account equity, scaled and then clipped:

$$r_t = \text{clip}\left(\lambda \ln \frac{E_t}{E_{t-1}}, -c, +c\right), \quad \lambda = 1000, \quad c = 1 \quad (30)$$

The log return is the natural choice for a trading objective because it is additive over time: the per-step rewards telescope to $\ln(E_T/E_0)$, so maximising their sum maximises terminal compounded wealth, and it is independent of account size. The scale factor is not cosmetic, and the reason is a result rather than a convenience. Risk-adjusted rewards of the differential kind described by Moody and Saffell, in which an online estimate of a Sharpe or Sortino ratio is accumulated, are scale-invariant: doubling every position leaves the ratio unchanged. An agent optimising such a reward receives no gradient pushing its position size upward, and in this environment it collapses to roughly 3% of the size available to it, producing a policy that is directionally sensible and financially pointless. A scale-sensitive reward is therefore necessary, and λ raises an hourly equity log return, typically of order 10^{-4} , to a magnitude the critic can resolve against its own approximation error.

The clip is a numerical safeguard of the kind introduced with the Deep Q-Network [8], applied after the encoding. Its effect here should be stated plainly rather than left implicit, because λ and c are not independent: together they clip the raw log return at $\pm c/\lambda = \pm 0.1\%$. On the delivered EUR/USD model the bound is reached on 38.2% of the 68 155 hourly steps, with a median absolute scaled reward of 0.70. For roughly two steps in five the agent is therefore told the direction of the equity move but not its magnitude. That is a real property of the objective, and it is reported here so that the learned behaviour in Section 5 is read against the reward the agent actually received.

4.2.4 Learning Algorithm: DDPG and its Adaptations

The algorithm is the Deep Deterministic Policy Gradient of Section 2.4, with the actor, critic and their delayed targets as described there, trained from a replay buffer with Ornstein-Uhlenbeck exploration noise. The equations of that section are not repeated. What follows are the six respects in which the

implementation departs from the original, each with the reason, and Table 15 sets every value against the published one.

Parameter	Original	This work	Status	Reason for the difference
Actor hidden layers	400×300	64×32	Changed	Noisy, non-stationary series supports less capacity
Discount factor γ	0.99	0.9995	Changed	Horizon matched to the holding period, months rather than days
Hidden normalisation	Batch	Layer	Changed	Single-observation inference must match batched training
Actor learning rate	10^{-4}	10^{-4}	Unchanged	
Critic learning rate	10^{-3}	10^{-3}	Unchanged	
Soft update rate τ	0.001	0.001	Unchanged	
Batch size N	64	64	Unchanged	
Replay capacity	10^6	10^6	Unchanged	
Critic weight decay	10^{-2}	10^{-2}	Unchanged	
Noise θ, σ	0.15, 0.2	0.15, 0.2	Unchanged	
Final layer init	$U[-3 \times 10^{-3}, 3 \times 10^{-3}]$	same	Unchanged	
Gradient clip	—	1.0	Added	Bounds a single update against a rare large error
Warmup transitions	—	3,000	Added	First updates drawn from a varied buffer
Actor penalty β	—	0.001	Added	Prevents saturation of the output tangent and constant-action collapse
Reward scale λ	—	1,000	Added	Lifts an hourly log return above the critic's approximation error
Reward clip c	—	1	Added	Numerical safeguard; bounds the raw log return at $\pm 0.1\%$

Table 15: Hyperparameters against the original Deep Deterministic Policy Gradient [3]. Three values were changed, five mechanisms were added, and the remainder are the published ones.

The first departure is the observation and the reward, described above.

The second is the size of the networks. The original uses hidden layers of 400 and 300 units for low-dimensional control tasks; here they are 64 and 32. A 38-dimensional observation over a noisy, non-stationary series supports far less capacity than a physics simulator with a clean reward, and larger networks fitted the training window without transferring. The third is the discount factor, raised from 0.99 to 0.9995. At an hourly step $\gamma = 0.99$ gives a horizon of about a hundred bars, roughly four days, which is shorter than the positions this strategy is intended to hold. At 0.9995 the horizon extends to some two thousand bars, on the order of three months, which matches the regime indicators in the observation.

The fourth is normalisation inside the networks. The original applies batch normalisation to the hidden layers; this implementation applies layer normalisation instead. The reason is deployment. Batch normalisation behaves differently on a single sample than on a training batch, because it depends on running statistics accumulated during training, and the agent must act on exactly one observation at a time when it runs on a live account. Layer normalisation normalises across the features of one sample, so the computation performed during training is the computation performed in deployment, and the target networks do not carry a second set of running statistics that must be kept synchronised.

The fifth is gradient clipping. The original specifies none; here the gradient norm of both the actor and the critic is clipped to 1 before each optimiser step. This bounds the size of a single update and prevents a rare large temporal-difference error from moving the parameters far enough to destabilise the critic. It is a stability measure and nothing more; in particular it does not address the collapse discussed next, which is caused by an absence of gradient rather than an excess of it.

The sixth is a penalty on the actor. This is the one change that alters the objective rather than its optimisation, and it exists because the algorithm as published does not train on this problem. The actor's final layer applies a hyperbolic tangent to a pre-activation $u(s)$ to bound the action, and the derivative of that function vanishes as $|u|$ grows. If the policy is driven into that region it stops responding: the action

saturates at ± 1 , no gradient flows back, and the agent holds one constant position for the remainder of training. The remedy is to penalise the magnitude of the pre-activation directly, which keeps the policy inside the responsive region of the tangent. Figure 6 shows where in the network that penalty acts:

$$L_{actor} = -\frac{1}{N} \sum_i \mathcal{Q}(s_i, \tanh u(s_i) | \theta^Q) + \frac{\beta}{N} \sum_i u(s_i)^2 \quad (31)$$

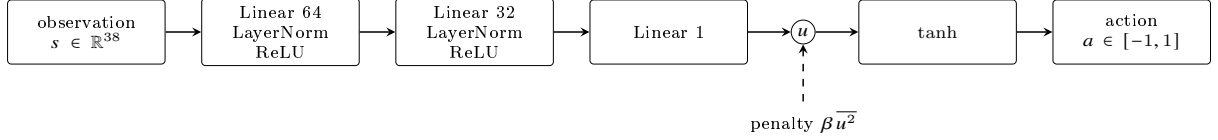


Figure 6: The actor. Layer normalisation replaces the batch normalisation of the original, so a single observation is processed exactly as a training batch is. The penalty of Equation 31 acts on the pre-activation u , before the tangent, which is where saturation would otherwise remove the gradient.

with $\beta = 0.001$. Setting $\beta = 0$ recovers the original actor loss exactly, so the change is a strict extension rather than a substitution. Together with the scale-sensitive reward, this is what prevents policy collapse; the two mechanisms address different halves of the same failure, the reward supplying a gradient that rewards larger positions and the penalty keeping the actor in the region where that gradient can act. Everything else follows the published algorithm. The learning rates, the soft update rate, the batch size, the replay capacity, the exploration noise parameters, the critic weight decay and the layer initialisation are the values given in the original work, and a warmup period of 3000 transitions is used before learning begins so that the first updates are drawn from a buffer with some variety in it rather than from the first few correlated steps of an episode.

4.3 Experimental Procedure: Training, Selection and Deployment

An agent design does not by itself produce a result. Training is stochastic, so the same configuration run twice gives two different policies, and a procedure is needed that says which of them is reported and on what grounds. This subsection describes that procedure, the criteria applied at each stage, and the two deployment decisions that sit between the learned policy and the orders actually sent.

4.3.1 Training Procedure: Seeds, Episodes and Data Splits

The eleven-year window is divided once into three consecutive periods. Writing the window as $[T_0, T_3)$ with T_0 the first of January 2015 and T_3 the first of January 2026:

$$\underbrace{[T_0, T_1)}_{\text{training, 9 years}} \quad \underbrace{[T_1, T_2)}_{\text{validation, 1 year}} \quad \underbrace{[T_2, T_3)}_{\text{testing, 1 year}} \quad (32)$$

Weights are updated on the training period only. The validation period decides which state of the network is kept. The testing period is visited once, after every other decision has been taken. Within that split, a single training run proceeds as follows. A seed fixes the network initialisation, the exploration noise stream and the order in which the replay buffer is sampled, and so determines the entire run. The agent then performs 30 episodes, an episode being one pass over the training period. Weights are not reset between episodes: the run is a single trajectory of 30 passes, and the network at the start of episode $n + 1$ is the network at the end of episode n . After each episode the agent is evaluated once over the validation period with learning disabled, and that evaluation is reduced to a single number by the Calmar ratio, its annualised return divided by its maximum drawdown. Writing f_n for that value and g_n for an eligibility test defined below, the state retained after episode n is

$$\theta^* \leftarrow \theta_n \quad \text{if} \quad (g_n \wedge \neg g^*) \vee (g_n = g^* \wedge f_n > f^*) \quad (33)$$

which keeps an eligible episode over an ineligible one regardless of score, and otherwise keeps the higher score within the same eligibility class. The saved state is a copy: the run continues training from θ_n , not from θ^* , so a later episode is never restarted from an earlier checkpoint. The eligibility test g_n rejects policies that are degenerate rather than merely poor. An episode qualifies only if it opened at least ten positions, spent at least three hundred bars on each side of the market, and held its weaker side for at least 30% of the time it held its stronger one. A policy that never trades, or that is long for the entire period, can produce a respectable Calmar ratio on a trending window while having learned nothing about when to hold a position, and the test removes it from consideration before the score is compared. Between 128 and 256 seeds were run for each currency pair, differing in nothing but the seed. Figure 7 shows how the three stages fit together.

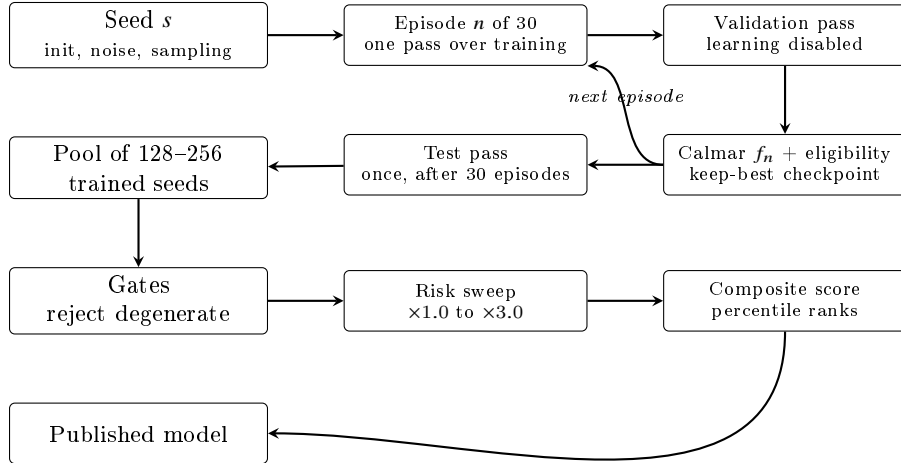


Figure 7: The three stages of the procedure. The Calmar ratio chooses among episodes within one seed; the composite score of Section 4.3 chooses among seeds.

One property of this arrangement must be stated explicitly, because it limits what the results in Section 5 can claim. The split of Equation 32 governs the training of an individual seed. The choice of which seed to publish, described next, is made on performance measured over the whole eleven-year window, and the performance figures reported in this work are measured over that same window. The testing period is therefore held out from the training of each candidate and from the choice among its episodes, but not from the choice among candidates, and the headline returns are not out-of-sample estimates. This is the weakest point in the method and it is not disguised anywhere in what follows.

4.3.2 Model Selection: Gates and Composite Ranking

Once the seeds for a pair have been trained, each is evaluated over the full window under the cost model of Table 13, and one is chosen. The choice is made in two steps, rejection followed by ranking, because the two questions are different: whether a policy is admissible at all, and which of the admissible ones is best.

Four gates reject a candidate outright. Its return must be positive. Its weaker direction must account for at least 10% of its exposure, which removes policies that are long or short throughout. Its mean holding period must be at least twenty-four bars, which removes policies whose signal is noise. And its regime score, the fraction of market movement during which it was positioned correctly, must exceed the score its own policy achieves when its position sequence is rotated in time. That last gate deserves a note, because a fixed threshold was tried first and does not work. A rotated policy holds the same positions for the same durations but at the wrong moments, so its score measures what that policy’s autocorrelation and holding structure would earn by luck alone. Measured across the seven pairs those

nulls range from 48 to 70, so a fixed bar of, say, 55 would be below chance on one pair and needlessly severe on another. The null is therefore computed per model rather than assumed.

Candidates surviving the gates are ranked by a composite of percentile ranks within their own pool. For a metric m and a pool of K candidates, the rank of candidate i is

$$R_m(i) = 100 \cdot \frac{|\{j : v_m(j) < v_m(i)\}|}{K - 1} \quad (34)$$

where v_m is negated for metrics that are better when smaller, such as drawdown. The metrics are divided into three groups, listed in Table 16, and the score is the unweighted mean of the three group means:

$$S(i) = \frac{1}{3} \sum_{G \in \{\text{money}, \text{safety}, \text{behaviour}\}} \frac{1}{|G|} \sum_{m \in G} R_m(i) \quad (35)$$

Group	Metrics, each entering as a percentile rank within the candidate pool
Money	Annualised return, Calmar ratio, Sterling ratio
Safety	Sharpe ratio, Sortino ratio, inverse maximum drawdown, inverse downside volatility
Behaviour	Two-sidedness (the weaker exposure side as a fraction of half), regime edge over the model's own rotation null, mean holding period, longest holding period

Table 16: Components of the composite selection score of Equation 35. The three groups carry equal weight, so return contributes one ninth of the total rather than dominating it. The ratios are those defined in Section 2.1.5.

Two design decisions are embedded in Equation 35. Ranks are relative to the pool rather than absolute, so no threshold can exclude a candidate that is simply better than everything around it. And the three groups carry equal weight, so a policy cannot win on return alone: money, safety and behaviour each contribute a third, and a candidate with the highest return in its pool but no two-sidedness and no regime edge scores poorly on two groups out of three. Each surviving candidate is also evaluated at several position sizes, from the reference risk of Equation 29 up to three times it. Scaling the risk fraction does not change the learned policy, only the size of the positions it takes, and the level chosen for each pair is reported with the results.

4.3.3 Deployment: Decision Cadence and Rebalance Threshold

Two decisions separate the learned policy from the orders that are actually sent, and between them they account for more of the delivered performance than any hyperparameter.

The first is the decision cadence. The agent observes every hourly bar, but it is permitted to act only once per calendar day. A key is formed from the timestamp of each bar and a new action is committed only when that key changes:

$$\kappa(t) = (\text{year}(t), \text{month}(t), \text{day}(t)) \quad (36)$$

The key is built from the calendar rather than from a count of bars, which makes it robust to the gaps and short sessions described in Section 4.1: a week containing a holiday still produces one decision per trading day rather than drifting out of alignment. The second is a rebalance threshold. A continuous action changes slightly at every decision, and acting on every such change would send a stream of tiny orders each paying the spread and a commission. An order is therefore sent only when the change in the target position is large enough to be worth its cost:

$$|V_{\text{target}} - V_{\text{current}}| \geq \max(V_{\text{min}}, \eta |V_{\text{ref}}|), \quad \eta = 0.20 \quad (37)$$

where V_{min} is the broker's minimum volume. The threshold is a fraction of the reference volume rather than of the position currently held, so it does not vanish as a position is reduced. Table 17 isolates the

contribution of each decision, measured on one model with nothing retrained. The comparison is between four configurations of the same weights, so the differences are attributable to the deployment layer alone.

Cadence	Threshold	Commission	Return	Sharpe	Max drawdown
Daily	—	none	+42.48%	0.319	24.4%
Hourly	—	none	−38.57%	−0.308	50.5%
Hourly	—	3.5 points	−64.70%	−0.732	70.5%
Daily	0.20	3.5 points	+34.02%	0.275	23.5%

Table 17: Contribution of the two deployment decisions, measured on one EUR/USD model over the full window with nothing retrained. All four rows are the same weights under different execution settings, so the differences are attributable to the deployment layer alone. This model precedes the seven-pair campaign and serves here only to isolate the effect.

Two things follow from that table. Moving from hourly to daily decisions is worth some eighty percentage points, turning a losing configuration into a profitable one, because an hourly cadence acts on noise the eleven-year horizon of the discount factor was never meant to capture. And the rebalance threshold recovers roughly twelve percentage points under realistic costs while reducing drawdown rather than increasing it, which is unusual: commission alone costs over twenty percentage points without the filter and about six with it. A continuous action space is what makes the strategy expressive, and this filter is what makes it affordable.

4.3.4 Worked Example: One Position From Decision to Settlement

The preceding subsections define each component separately. This one follows a single position through all of them, using a trade taken by the delivered EUR/USD model, so that the arithmetic can be checked rather than assumed.

At 02:00 on 30 January 2019 the daily decision key of Equation 36 changed, and the agent was consulted for the first time that day. It received the 38-value observation of Section 4.2, built from the bar that had just closed, and returned a positive action. The reference volume of Equation 29 at that moment, risking 1% of the balance over a stop of 1.5 average true ranges, gave a target of 12,000 units of the base currency once the action had scaled it and the result was rounded to the broker’s volume step. The change from the position then held exceeded the threshold of Equation 37, so an order was sent. The tick recorded at that instant quoted 1.14392 ask against 1.14387 bid, a spread of five points. The buy was filled at the ask. Twenty-four hours later the key changed again, the agent’s action reversed the sign, and the position was closed at the bid then quoted, 1.14931. The profit follows from those two prices and the volume, and is expressed in the account currency at the conversion rate recorded on the closing tick:

$$12,000 \times (1.14931 - 1.14392) = 64.68 \text{ USD} \longrightarrow 56.27 \text{ EUR} \quad (38)$$

The spread is already inside that figure. It was paid at entry by filling at the ask rather than the mid, and again at exit by filling at the bid, which is what makes the round trip cross it exactly once. The commission is charged separately at both ends:

$$2 \times 3.5 \times 10^{-5} \times 12,000 = 0.84 \text{ USD} \longrightarrow 0.72 \text{ EUR} \quad (39)$$

The swap is zero, the position having been held overnight on a swap-free account. The position therefore settled at $56.27 - 0.72 = 55.55$ EUR, which is the figure recorded in the exported trade record. One trade says little. Across all 1,777 trades the delivered EUR/USD model took over the eleven years, the same accounting gives a gross profit of 8,733.62 EUR, a commission charge of 1,669.50 EUR and a swap charge of exactly zero, for a net of 7,064.12 EUR. Commission alone consumed 19.1% of the gross profit, and that is before the spread, which is not separable from the gross figure because it is embedded in every fill. A result reported without these charges would be describing a different strategy.

Everything described here was fixed before any result was seen: one architecture, one observation, one reward, one selection rule, applied unchanged to seven instruments. That constraint is what makes the outcomes comparable across pairs.

It also makes them falsifiable. A design tuned until it works on a single instrument can always be made to work on that instrument, and nothing in its results distinguishes a strategy from a description of that instrument’s history. Applying the same design to seven leaves it nowhere to hide, and the results in Section 5 include a pair on which it does not work.

5 Experimental Results

This section reports what the design of Section 4 produced. Seven agents were trained, one per major currency pair, and each was evaluated over eleven years under the cost model of Table 13. Every figure below traces to a stored run that can be reopened, and the analysis was recomputed from those runs rather than copied from the training logs.

The order of the section is deliberate. Returns come first, then the checks that decide whether those returns mean anything, then the year that was held back, and finally the result of running the seven together.

5.1 Evaluation Protocol

The account, the cost model and the execution engine are described in Section 4.1 and are not repeated here. Three things are specific to the evaluation.

The window runs from 1 January 2015 to 1 January 2026, eleven years. It starts a year after the data begins so that the slowest indicator in the observation, a moving average over five thousand hourly bars, is fully warmed before the first decision.

Every result is measured from five different starting balances, of 9900, 10000, 10050, 10100 and 10200 euros. A trading strategy that depends on its entry point is not a strategy, and small changes to the opening balance shift the rounding of every position size and therefore the whole sequence of trades. Reporting one balance would hide that sensitivity, so the tables give the mean across the five and the spread around it.

Position size is a per-pair parameter, chosen by the sweep described in Section 4.3 rather than fixed in advance. Five pairs use the reference risk of Equation 29, NZD/USD uses one and a half times it, and USD/JPY uses a smaller fraction of it. Position size scales the size of every trade but does not change the policy that chooses them, so the ratios, the alpha and beta decomposition and the correlation structure are all unaffected by it.

5.2 Benchmarks and Performance Metrics

The benchmark is buying the pair at the start of the window and holding it to the end. It is the honest comparison for a directional strategy on a single instrument: it is available to anyone, it costs one spread, and any strategy that cannot beat it has not earned its complexity. The proposal named a random-signal baseline and a moving-average crossover instead; both were dropped because neither is a strategy a trader would actually hold as an alternative.

The reported metrics are those defined in Section 2.1.5: total and annualised return, the Sharpe, Sortino, Calmar and Sterling ratios, maximum drawdown and the number of trades. Table 18 states what each one measures. Ratios use a zero risk-free rate throughout, so they can be compared with each other across pairs without a rate assumption entering.

Four of them deserve a word on why they are all reported rather than one being chosen. Return says how much was made and nothing about the cost of making it. The Sharpe ratio divides return by total volatility, which penalises a strategy for moving upwards as much as for moving downwards. The Sortino ratio corrects that by counting only downside movement. Calmar and Sterling divide by drawdown instead of volatility, the first by the worst loss ever suffered and the second by the average one, so they answer the question an investor actually asks, which is how far the account fell rather than how much it wobbled. A strategy can look strong on one of these and weak on another, and where that happens in this work it is pointed out.

One point applies to every number in this section. Returns are net of the spread quoted at the moment of each trade and of the commission of the account in Table 13. They are therefore not comparable with results computed on bar closes with a fixed or absent spread, and they are lower than the same policies would show under those conditions.

Metric	Description
Net Profit	Total profit or loss generated
Profit Factor	Ratio of gross profit to gross loss
Commission and Swap	Costs affecting net profitability
Max Balance and Equity Drawdown	Maximum observed losses
Total Trades and Winning Trades	Indicators of activity and success
Max Consecutive Winning and Losing Trades	Consistency Indicators
Largest Winning and Losing Trades	Potential for significant gains or drawdowns
Average Trade	Measure of trading efficiency
Sharpe Ratio and Sortino Ratio	Measures of Risk-Adjusted Performance

Table 18: Summary of the performance metrics that will be used.

5.3 Results Across the Seven Majors

Table 19 gives the result for each pair, in alphabetical order. Total return over the eleven years ranges from +226.6% on USD/CAD, the strongest, to +15.7% on USD/JPY, the weakest. Every pair is profitable, and every pair is profitable from all five starting balances.

Pair	Mean	SD	Min	Max	Sharpe	Sortino	Calmar	Max DD	Trades
AUD/USD	102.5%	10.6	87.2%	117.1%	0.348	0.496	0.143	41.1%	1,561
EUR/USD	73.9%	3.3	69.5%	77.8%	0.387	0.544	0.106	46.8%	1,778
GBP/USD	124.1%	5.1	115.1%	129.1%	0.397	0.563	0.155	50.4%	2,516
NZD/USD	56.4%	3.8	52.2%	61.7%	0.368	0.531	0.187	23.3%	1,011
USD/CAD	226.6%	10.8	216.9%	246.0%	0.603	0.870	0.228	49.1%	2,213
USD/CHF	50.6%	5.2	43.4%	56.4%	0.802	1.552	0.492	8.2%	220
USD/JPY	15.7%	0.7	14.5%	16.5%	0.251	0.342	0.057	24.0%	1,438

Table 19: Result for each pair over 2015 to 2026. Return columns are the mean, standard deviation and range across the five starting balances of Section 5.1; every pair is profitable from all five. Ratios and drawdown are from the reference run. USD/CAD returns the most and USD/CHF is best on every risk measure.

One fact frames everything that follows and is easy to miss. **Five of the seven pairs lost value over the eleven years.** A trader who had simply bought and held would have finished down 26.1% on NZD/USD, 20.3% on USD/CHF, 18.2% on AUD/USD, 13.5% on GBP/USD and 2.9% on EUR/USD. Only USD/CAD, at +18.1%, and USD/JPY, at +30.7%, rose. The agents made money on all seven, which means that on five of them the return was earned against a falling market rather than alongside a rising one.

Pair	Market	Model	Excess	Positive Years
AUD/USD	-18.2%	+87.2%	+105.4 pp	8 of 11
EUR/USD	-2.9%	+70.6%	+73.4 pp	5 of 11
GBP/USD	-13.5%	+129.1%	+142.5 pp	7 of 11
NZD/USD	-26.1%	+60.1%	+86.1 pp	7 of 11
USD/CAD	+18.1%	+221.0%	+202.9 pp	8 of 11
USD/CHF	-20.3%	+54.8%	+75.2 pp	10 of 11
USD/JPY	+30.7%	+16.2%	-14.4 pp	5 of 11

Table 20: Each agent against buying and holding its own pair over the same window and under the same costs. Five of the seven pairs lost value over the eleven years, so on those the return was earned against a falling market. USD/JPY is the only pair whose benchmark rose substantially, and the only one the agent failed to beat.

The rest of this subsection takes each pair in turn, in alphabetical order. For each one the discussion covers what the market itself did, what the agent returned against it, how that return decomposes into policy and market exposure, what the agent paid to earn it, and how it behaved while doing so.

5.3.1 AUD/USD

The Australian Dollar fell 18.2% against the US Dollar over the eleven years. The agent returned +87.2%, which is 105.4 percentage points ahead of holding the pair, and it earned that from the short side: only 27.9% of its 1561 positions were long.

The decomposition is where this result needs care. Its beta of -2.013 is the largest negative exposure in the set, meaning the agent carried the falling market at roughly twice its size. Of its 8.18% mean yearly return, 4.96 points are alpha and 3.22 points are drift, so close to two fifths of what it earned came from the Australian Dollar falling rather than from choosing when to be short. The policy is genuinely profitable, but part of that profit would reverse in a decade where the pair rose.

Its trading is moderately active, a median holding period of 9.7 days at an average of 0.77 of reference size, with a win rate of 52.7%. It generated 10 689 euros of gross profit and paid 2 200 in commission, a 20.6% share that is the highest of the seven and a direct consequence of that turnover. The equity curve is uneven: eight of eleven years are positive, the best being 2024 at +72.4% and 81.6 points clear of the benchmark, and the worst 2017 at -22.5% . Its maximum drawdown is 41.1%.

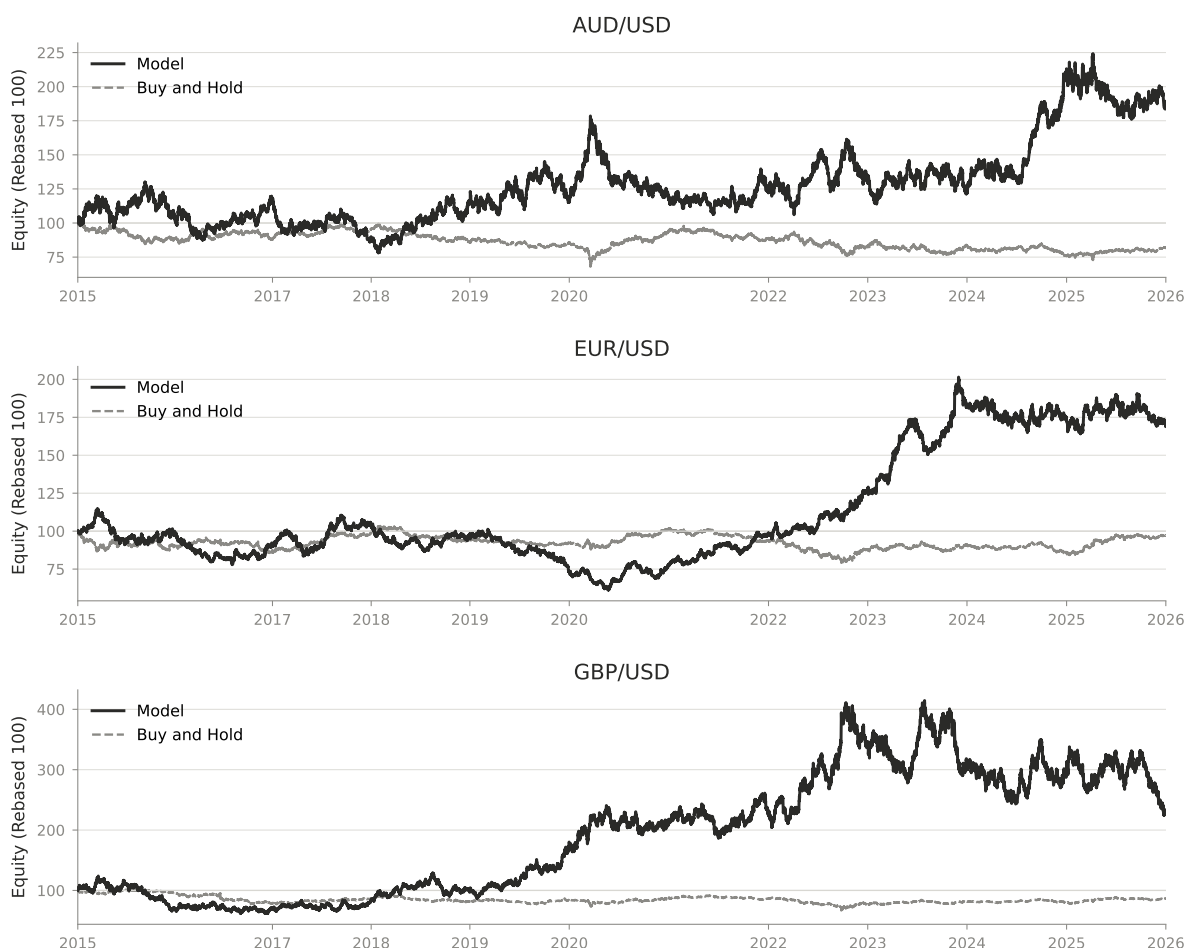


Figure 8: AUD/USD, EUR/USD and GBP/USD against buying and holding the same pair, rebased to 100. All three benchmarks finish below their starting level.

5.3.2 EUR/USD

EUR/USD is the cleanest demonstration in this work of return that is not market exposure. The pair finished almost exactly where it started, down 2.9% across eleven years, and the agent returned +70.6%. Its beta is +0.266 and its market exposure contributes 0.01 percentage points per year, so of a 6.63% mean yearly return, 6.62 points are attributable to the policy. There was no trend to ride; the return came from timing alone.

It trades the shortest holding period of the seven, a median of 3.9 days, at a modest 0.40 of reference size, and its exposure is close to balanced at 55.5% long across 1 778 positions. It won 51.9% of them. Gross profit of 8 734 euros became 7 064 after 1 669 euros of commission, a 19.1% share.

The equity curve, in Figure 8, shows why the headline figure needs qualifying in a different way from AUD/USD. Only five of eleven years are positive. The agent is roughly flat for the first six years of the window and produces almost all of its return between 2021 and 2023, when the pair moved in unusually wide ranges: +27.3%, +29.0% and +44.0% in those three years, against -25.3% in 2019. This is a policy that needs volatility to work, and its maximum drawdown of 46.8% reflects the periods when it did not get it.

5.3.3 GBP/USD

GBP/USD produced the highest alpha in the set at +11.64% per year, and it did so with a beta of -0.054 that is indistinguishable from zero. Its mean yearly return is 11.70%, of which 0.06 points come from market exposure: as close to pure policy as any result here. Sterling itself fell 13.5% over the window, so the agent's +129.1% sits 142.5 percentage points above the benchmark.

It is the most active model, 2 516 positions, and also holds them the longest among the six working agents at a median of 12.6 days, which means it runs many positions concurrently rather than trading quickly. It operates near full size at 0.89 and wins 51.3% of the time. Gross profit was 16 630 euros against 3 168 of commission, keeping 13 462.

Its returns concentrate in the years when sterling moved violently, which is visible in Figure 8 as two steep sections: +82.2% in 2019 and +52.0% in 2022. Section 5.4 examines the second of those in detail. The same aggression produces the largest single losing year in the set, -31.6% in 2015, and the deepest drawdown, 50.4%. Seven of eleven years are positive. This is the most volatile of the profitable models and a reader should weigh its alpha against a drawdown that halved the account.

5.3.4 NZD/USD

The New Zealand Dollar fell furthest of the seven, 26.1% over the window, and the agent returned +60.1% against it, an excess of 86.1 points. Like AUD/USD it worked mostly from the short side, 24.0% of positions long, with a beta of -0.708.

Its alpha of +2.79% per year is the smallest among the six profitable models, and with 1.83 points of beta return in a 4.62% mean return, a substantial minority of what it earned came from the market falling rather than from timing. On the alpha and beta reading it is the weakest of the six that worked.

Figure 9 shows it alongside USD/CAD and USD/CHF. What distinguishes it is restraint, and the risk numbers reward it. It takes the fewest positions of the active models, 1 011, holds an average of 0.24 of reference size, and produces a maximum drawdown of 23.3%, the second lowest in the set. Its Calmar ratio of 0.187 is better than four pairs that returned considerably more, which is the point of measuring return against drawdown rather than in isolation. Gross profit was 7 314 euros and commission 1 424, a 19.5% share. Seven of eleven years are positive, the best 2022 at +14.5% and the worst 2016 at -6.4%; this is the steadiest of the moderately-sized models.

5.3.5 USD/CAD

USD/CAD returned +226.6%, by a wide margin the largest total in this work, and it is also the result that requires the most careful reading. Its beta of +2.393 is the highest of the seven and the pair itself

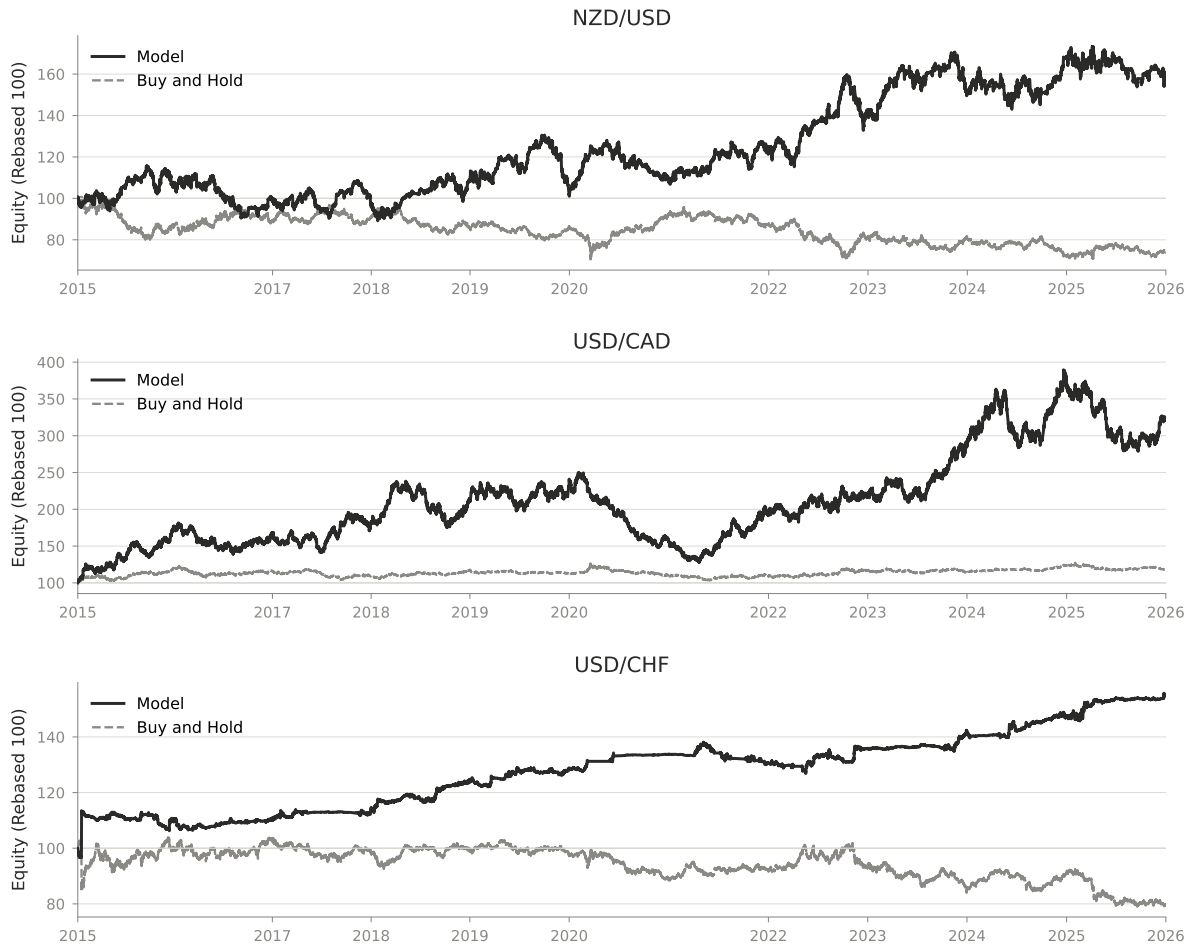


Figure 9: NZD/USD, USD/CAD and USD/CHF against buying and holding the same pair, rebased to 100. USD/CAD is the only one of the three whose benchmark rises.

rose 18.1%, so the agent was carrying a rising market at more than twice its size. Of a 14.23% mean yearly return, 4.21 points are beta return and 10.02 are alpha.

That alpha is real and it is the second highest in the set, so this is not a model that merely looks good because its market went up. But leverage is symmetric, and the year-by-year record shows both sides of it: +71.7% in 2015, its best year and 52.7 points clear of the benchmark, against -34.8% in 2020, its worst. Its maximum drawdown is 49.1%, and recovering from a 49% drawdown requires a 96% gain. Eight of eleven years are positive.

It is the second most active model at 2213 positions with a median hold of 5.7 days, runs at 0.79 of reference size, and has the highest win rate in the set at 55.0%. It also earned the most in absolute terms, 25577 euros gross, and paid 3432 in commission for a 13.4% share, lower than the four pairs above it because its profit per position was larger.

5.3.6 USD/CHF

USD/CHF returned +54.8%, fifth of seven on return and first on every measure that accounts for risk. Its Sharpe ratio of 0.802, Sortino of 1.552 and Calmar of 0.492 are the highest here, and its maximum drawdown of 8.2% is a sixth of the worst. The Swiss Franc strengthened over the period so the pair fell 20.3%, and with a beta of -0.082 the agent took essentially none of that movement: of a 4.07% mean

yearly return, 3.91 points are alpha.

Its behaviour explains every one of those numbers. It takes 220 positions in eleven years, roughly one every eighteen days, and holds an average of 0.09 of its reference size, a tenth of what it is permitted. Because it trades so little it pays almost nothing to trade: 130 euros of commission against 5 587 of gross profit, a 2.3% share where the set average is 16.0%. It keeps 97.7% of what it earns while AUD/USD keeps 79.4%.

It is positive in ten of eleven years, the most consistent record in this work, with a worst year of -2.4% . Its win rate of 49.8% is the lowest of the seven, which is worth pausing on: the best model in the set is wrong slightly more often than a coin, and is the best anyway because of how it sizes and holds the positions it gets right. It is also the only model that made money in the held-out year of Section 5.8. On return alone it would have placed fifth and been passed over; the selection criteria of Section 4.3 exist to prevent exactly that.

5.3.7 USD/JPY

USD/JPY is the failure of the set, and it is the only pair whose benchmark rose substantially. The pair gained 30.7% over the window while the agent returned +15.7%, finishing 14.4 percentage points behind an investor who bought it and did nothing. Its beta of +1.012 and correlation of +0.946 with the pair's own yearly move describe what it became: a long position with extra steps and extra costs. Its alpha is -1.08% per year, the only negative figure in Table 24.

Every position it takes is long, its median holding period is measured in years rather than days, and its average action is 0.98, meaning it holds essentially full size continuously and has stopped making a sizing decision at all. It earned 1 761 euros gross, the least of the seven, and paid 155 in commission. Five of eleven years are positive.

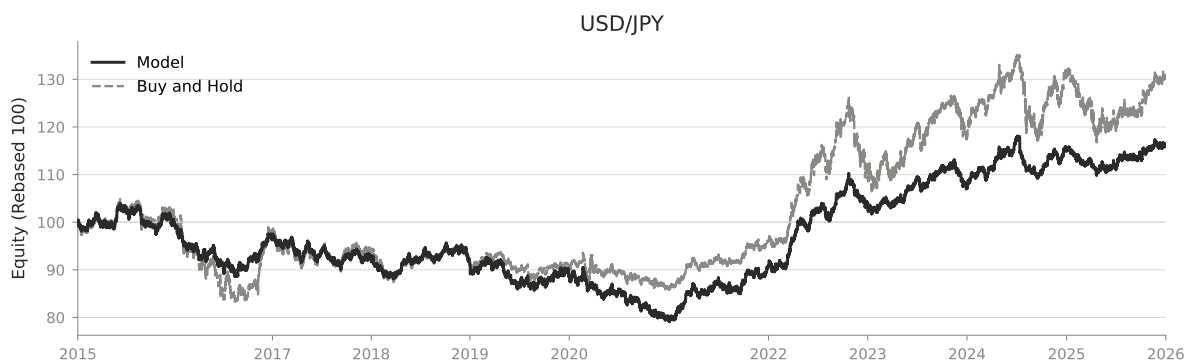


Figure 10: USD/JPY against buying and holding the pair. The two lines track each other closely, which is the visual form of a beta of +1.012 and the reason this result is reported as a failure.

Figure 10 makes the point better than the statistics do: the model and the benchmark move together, with the model slightly below. Section 5.9 sets out the mechanism and what was tried against it.

5.4 Trading Behaviour: How the Agents Actually Trade

A return figure says whether an agent made money. It says nothing about how, and that matters here for a specific reason. The central claim of this work is that a continuous action lets one decision set both the direction and the size of a position. If the agents had learned to use only the extremes of that action, they would be discrete traders with extra machinery, and the claim would be empty. This subsection examines what they actually did.

5.4.1 Use of the Continuous Action

The agent emits an action $a \in [-1, 1]$ and holds $|a|$ of the reference size defined in Equation 29. Figure 11 shows how that value is distributed across the whole evaluation window for each pair.

Pair	Positions	Long	Median Hold	Longest Hold	Win Rate	Mean $ a $
AUD/USD	1,561	27.9%	9.7 d	227 d	52.7%	0.77
EUR/USD	1,778	55.5%	3.9 d	43 d	51.9%	0.40
GBP/USD	2,516	46.9%	12.6 d	149 d	51.3%	0.89
NZD/USD	1,011	24.0%	5.9 d	119 d	52.8%	0.24
USD/CAD	2,213	65.6%	5.7 d	97 d	55.0%	0.79
USD/CHF	220	34.2%	8.0 d	345 d	49.8%	0.09
USD/JPY	1,438	100.0%	2,120 d	4,004 d	50.7%	0.98

Table 21: Trading behaviour over the evaluation window. Mean $|a|$ is the average absolute action, that is the average fraction of the reference position size held while in the market. Win rate is the share of closed positions with a positive net result after costs.

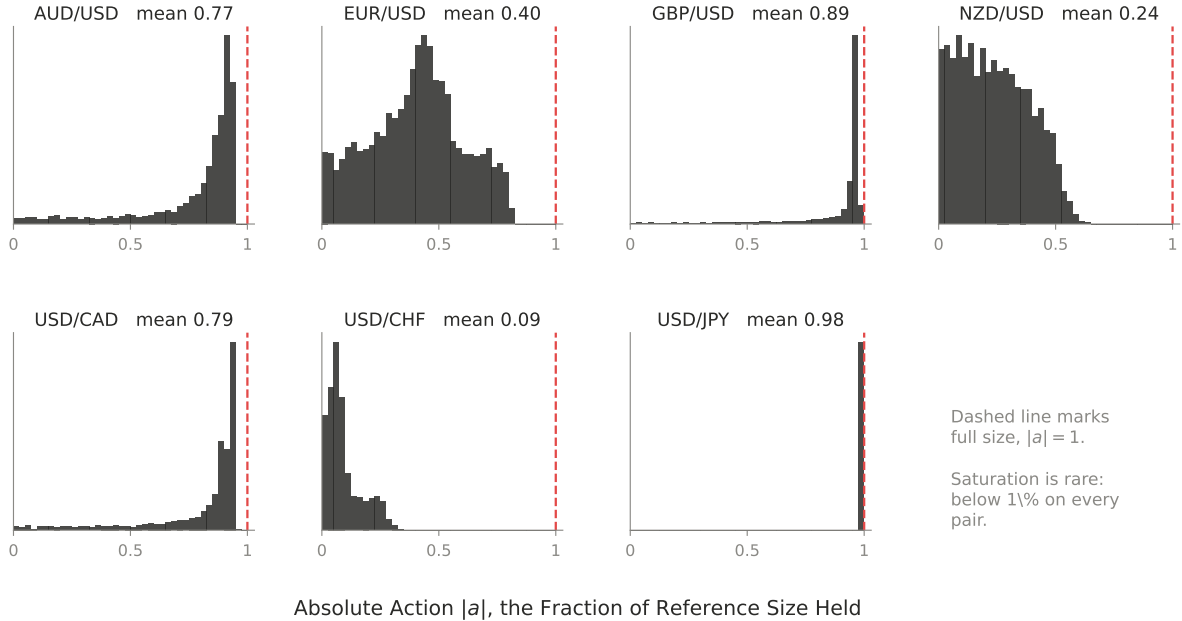


Figure 11: Distribution of the absolute action across the evaluation window. The dashed line marks full size. If the agents had reduced the continuous action to a discrete one, these histograms would be concentrated against that line; instead the mass sits across the interior of the range.

The action does not saturate. Fewer than one percent of decisions on any pair sit above $|a| = 0.98$, and on six of the seven pairs the figure is zero to one decimal place. The mean absolute action ranges from 0.09 on USD/CHF to 0.98 on USD/JPY, so the agents settled on very different operating sizes from the same design and the same reference risk.

That spread is itself informative. USD/CHF holds roughly a tenth of the size available to it, which is why its maximum drawdown is 8.2% while its Sharpe ratio is the highest in the set: it found a small edge and sized it accordingly rather than pressing it. NZD/USD, at 0.24, behaves similarly. USD/JPY sits at the opposite extreme with 0.98, which is the numerical signature of the failure described in Section 5.9: an agent that always holds full size in one direction has stopped making a sizing decision at all.

The sizing is adaptive in a second sense that the histogram does not show. The reference size itself is inversely proportional to the average true range, so a constant action produces a smaller position

when volatility rises. The action and the volatility scaling compose, and the position actually held is the product of the two.

5.4.2 Direction, Size and Outcome Together

Figure 12 follows EUR/USD through three years, showing the price, the action the agent emitted, the position it held as a result, and the equity that followed.

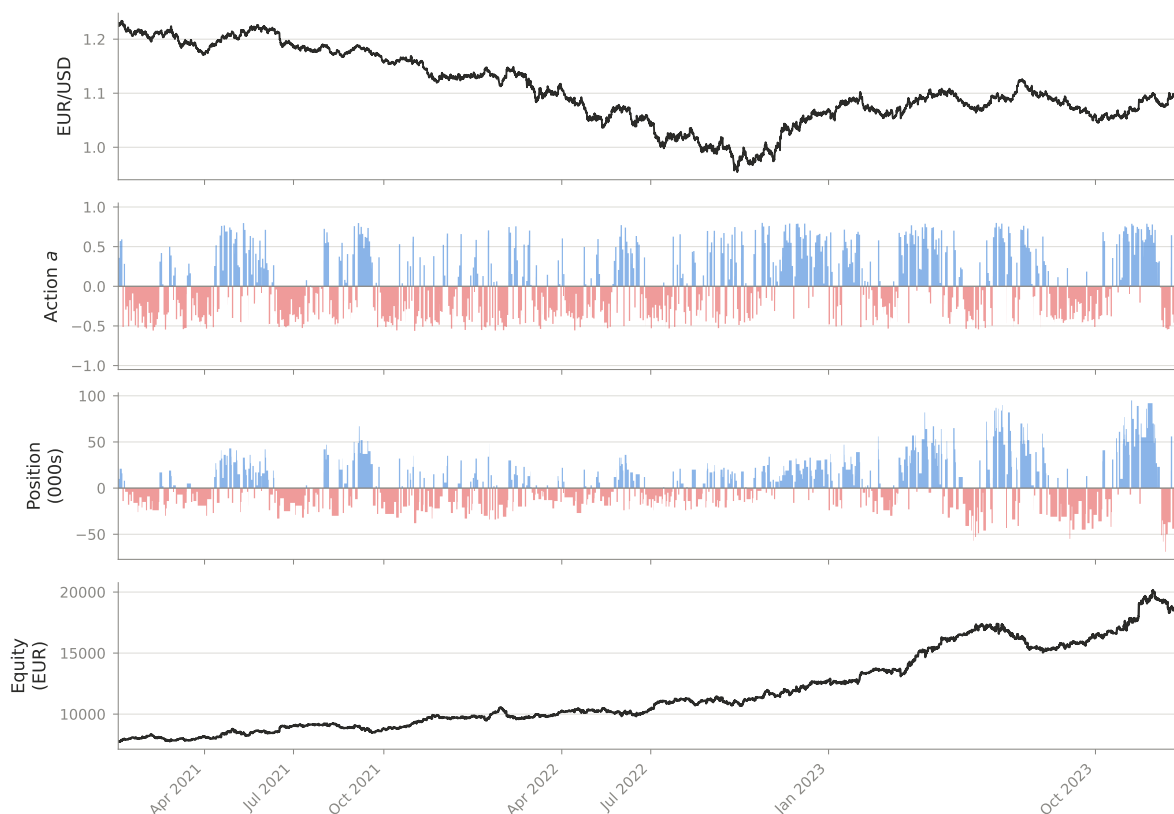


Figure 12: EUR/USD from 2021 to 2024. From top: the price; the action, positive for long and negative for short; the resulting net position in thousands of units; and account equity. The action changes sign well before the position does, because a change is only acted on when it exceeds the threshold of Equation 37.

Three behaviours are visible. The action moves continuously rather than switching between fixed levels, and its magnitude varies as much as its sign. The position follows the action but in coarser steps, because the rebalance threshold suppresses small adjustments; long stretches of constant position correspond to periods where the agent kept revising its target by less than the threshold. And the sign changes are not frequent: over a three-year window the agent reverses direction a few dozen times, which is consistent with the daily decision cadence rather than with an agent reacting to every bar it observes.

5.4.3 Trend Identification and Reversal

The clearest illustration of the agents holding a position through a sustained move is GBP/USD in the second half of 2022, shown in Figure 13. Sterling fell from 1.22 to below 1.04 over roughly ten weeks, a decline of more than fifteen percent, with the sharpest part following the United Kingdom fiscal announcement of late September.



Figure 13: GBP/USD from mid-July to mid-October 2022. Background shading marks the direction of the net position, red for short and blue for long. The lower pane gives the size of that position. The annotated trade is the single most profitable position of the eleven years on this pair.

The agent was short for almost the entire decline. It held short exposure of between fifty and one hundred thousand units through the fall, added to it as the move developed, and reduced it into the low. The annotated position was opened at 1.1396 on 12 August and closed at 1.0403 on 26 September, forty-five days later, for a net profit of 2073 euros after costs. It is the largest single position of the eleven years on this pair.

Two details are worth pointing out because they show discrimination rather than luck. The agent was briefly long at the end of July, at the local high before the decline began, and it was long again in the days immediately after the bottom, both times at small size. It did not hold the short indefinitely. Once the price recovered above 1.12 in early October the position was reduced and then reversed, which is the behaviour of an agent responding to the state it observes rather than one committed to a direction.

5.4.4 Holding Periods and Two-Sided Exposure

Table 21 gives the trading statistics for each pair and Figure 14 shows two of them directly. Median holding periods run from four days on EUR/USD to nearly thirteen on GBP/USD, with the longest individual positions measured in months. These are swing traders, not scalpers, which is what the daily decision cadence of Section 4.3 was intended to produce.

The long and short shares differ substantially between pairs, from 24% long on NZD/USD to 66% on USD/CAD, with EUR/USD and GBP/USD close to balanced. None of the six working agents is one-sided, which is a consequence of the design rather than an accident: the eligibility test of Section 4.3 rejected any episode whose weaker side fell below a threshold, so a one-sided policy could not be selected however profitable it looked. USD/JPY is the exception at 100% long, and it is the exception precisely because that gate could not be satisfied on that pair by any of the 448 seeds that were tried.

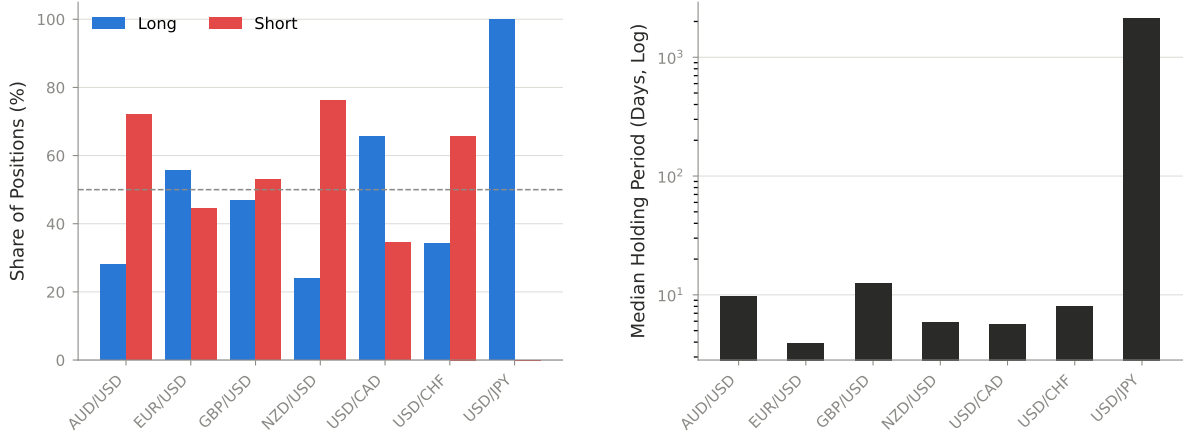


Figure 14: Left: the share of positions taken long and short. Right: the median holding period in days, on a logarithmic scale. USD/JPY is long in every position it takes and holds for years, which places it off the scale of the other six.

5.4.5 Where the Edge Comes From

One statistic in Table 21 is worth isolating. The win rate, the fraction of positions closed at a profit, lies between 49.8% and 55.0% across the seven pairs. Six of the seven are within three points of a coin flip, and the best model in the set by every risk measure, USD/CHF, has the lowest win rate of all at 49.8%.

The agents are therefore not profitable because they are right more often than they are wrong. They are profitable because of what they do with a position once it is open: the size held when the position moves favourably is larger than the size held when it does not, and positions that work are held for longer than positions that do not. That is what a continuous action buys, and it is not available to an agent that can only choose a direction. It also explains why USD/CHF can be the strongest model in the set while being wrong slightly more often than a coin: it is small and patient, and the few positions it holds at size are the ones that worked.

5.4.6 What the Trading Costs

Because the engine charges the spread quoted at each fill and a real commission schedule, the difference between what the policies earned and what the account kept can be measured rather than estimated.

The commission is recorded directly on every trade. The spread is not, because it is paid inside each fill rather than deducted after it: a buy is filled at the ask and a sell at the bid, so the cost is already embedded in the entry and exit prices. It can nonetheless be recovered exactly, because the two charges scale in the same way. Commission is levied at 3.5 points per side, seven points for a round turn, and the spread is crossed once per round turn. Both are proportional to the traded volume and both are converted into the account currency at the same rate, so for any position

$$C_{spread} = |C_{commission}| \cdot \frac{s}{7} \quad (40)$$

where s is the spread in points at the moment of the trade. Applying Equation 40 to every position, with s taken from the recorded quotes for the month in which the position was opened, gives Table 22.

The seven agents earned 85 039 euros before costs and kept 64 112. Of the difference, 8 748 euros went to the spread and 12 179 to commission, so **costs consumed 24.6% of trading profit**. Commission is the larger of the two charges here, which is a consequence of trading a raw-spread account: the spread is narrow by design and the broker recovers its margin explicitly instead. On an account quoting wider spreads with no commission, the same trades would have paid a similar total in a different proportion.

Pair	Before Costs	Spread	Commission	Net	Cost Share	Mean Spread
AUD/USD	11,851	-1,162	-2,200	8,488	28.4%	3.70 pts
EUR/USD	9,372	-638	-1,669	7,064	24.6%	2.68 pts
GBP/USD	19,086	-2,456	-3,168	13,462	29.5%	5.43 pts
NZD/USD	8,552	-1,238	-1,424	5,890	31.1%	6.09 pts
USD/CAD	28,272	-2,695	-3,432	22,145	21.7%	5.50 pts
USD/CHF	5,735	-148	-130	5,457	4.8%	7.93 pts
USD/JPY	2,172	-411	-155	1,606	26.1%	18.53 pts
Total	85,039	-8,748	-12,179	64,112	24.6%	—

Table 22: What the seven agents earned before costs and what they kept, in euros. Cost share is the spread and commission together as a fraction of profit before costs. The mean spread column is the commission-weighted average spread paid, in points, which differs from the pair’s average spread because it weights the moments the agent actually traded.

The swap column is absent from Table 22 because it is zero on every pair. On a swap-free account no overnight financing is charged, so that is the exact figure rather than an approximation, and it matters at this holding horizon: agents holding positions for a median of four to thirteen days would otherwise pay financing on several hundred nights each.

The variation between pairs follows turnover almost exactly, and it is large. USD/CHF takes 220 positions and surrenders 4.8% of its pre-cost profit, keeping more than nineteen euros in twenty. NZD/USD surrenders 31.1% and GBP/USD 29.5%. The same quality of policy therefore delivers very different net results at different trading frequencies, which is why the selection criteria of Section 4.3 reward long holding periods directly rather than trusting the return figure to account for them, and why the rebalance threshold of Equation 37 contributes as much as Table 17 indicates.

One consequence deserves emphasis because it bears on how the rest of this section should be read. Every return quoted in this work is net of both charges. A study reporting the same policies on bar closes with no spread and no commission would have reported the pre-cost column of Table 22, which is a third larger, and it would have been describing a strategy that no account could have traded.

Recovering the spread through Equation 40 works but is indirect, and it relies on the commission being non-zero. The engine computes the spread charge internally when it opens a position and does not currently carry it into the exported trade record as its own field. Recording it there, alongside commission and swap, is a straightforward improvement and is noted in Section 6.3.

5.5 Path Robustness

Each pair was evaluated from the five starting balances described in Section 5.1. The result is the same in every case: all seven pairs are profitable from all five entry points, thirty-five runs in total, with no exceptions. Table 23 gives the individual figures.

Pair	9,900	10,000	10,050	10,100	10,200	Mean	SD	Relative SD
AUD/USD	94.2%	87.2%	117.1%	108.3%	105.7%	102.5%	10.58	10.3%
EUR/USD	69.5%	70.6%	76.5%	77.8%	75.1%	73.9%	3.27	4.4%
GBP/USD	126.5%	129.1%	127.9%	122.0%	115.1%	124.1%	5.11	4.1%
NZD/USD	61.7%	60.0%	54.4%	53.7%	52.1%	56.4%	3.76	6.7%
USD/CAD	230.4%	221.0%	218.5%	216.9%	246.0%	226.6%	10.81	4.8%
USD/CHF	56.4%	54.8%	43.4%	52.6%	45.5%	50.6%	5.16	10.2%
USD/JPY	15.9%	16.2%	16.4%	15.7%	14.5%	15.7%	0.69	4.4%

Table 23: Total return from each of the five starting balances, in euros. All thirty-five runs are profitable. Because broker volumes are discrete, a different opening balance rounds positions differently and produces a genuinely different sequence of trades rather than a rescaled copy of the same one.

The reason this test is worth running is not obvious, so it is worth setting out. A change of one hundred euros in the opening balance looks like a change of scale, and if position sizes were continuous

it would be exactly that: every trade would be one percent larger, the equity curve would be a scaled copy, and the percentage return would be identical. Sizes are not continuous. A broker accepts volume in discrete steps, so the reference volume of Equation 29 is rounded before an order is sent. A slightly larger balance can round a position up where a smaller one rounds it down, which changes the equity available for the next decision, which changes the size of the next position, and so on. Two runs differing only in their opening balance are therefore not the same run rescaled; after a few dozen trades they hold different positions at different times.

That the seven pairs remain profitable across all five paths, with returns clustered rather than scattered, says the results do not depend on one particular sequence of roundings. The relative standard deviation, the standard deviation divided by the mean, sits between 4.1% and 10.3%. GBP/USD is the tightest at 4.1% despite being the most active model, and AUD/USD the widest at 10.3%.

Two observations follow from the spread of the individual figures. The pairs with the widest relative spread, AUD/USD at 10.3% and USD/CHF at 10.2%, are not the pairs with the largest returns; dispersion tracks how close the agent’s typical position size sits to a rounding boundary rather than how much it makes. And no pair has a path that turns a profit into a loss, or even into a materially different conclusion: the worst path on USD/CAD still returns +216.9% and the best on USD/JPY still returns +16.4%, so the ranking between pairs is stable across all five.

What this protocol does not measure should be stated as clearly as what it does. It varies the entry path, not the training run. Each published model is one seed drawn from a pool of between 128 and 256, and the variation across that pool is a different and larger source of uncertainty, addressed in Section 5.7 and again in Section 5.8. A result can be perfectly stable across entry paths and still be the product of a fortunate draw.

5.6 Alpha and Beta Decomposition

A currency pair moves on its own. An agent that is long a pair which happens to rise will show a profit that has nothing to do with the policy it learned, and an agent that is short a pair which happens to fall will show the same. Total return cannot tell the two apart, so the results are decomposed into the part explained by the market and the part that is not.

For each pair the yearly return of the model is regressed on the yearly move of the pair itself, over the eleven years from 2015 to 2025. Writing r_i for the model return in year i and m_i for the market move in the same year, ordinary least squares gives

$$\beta = \frac{\sum_i (m_i - \bar{m})(r_i - \bar{r})}{\sum_i (m_i - \bar{m})^2}, \quad \alpha = \bar{r} - \beta \bar{m} \quad (41)$$

β is how much of the market’s movement the model carries, and α is the average yearly return left once that exposure is removed. The mean return then splits into an active part and a passive one:

$$\bar{r} = \underbrace{\alpha}_{\text{policy}} + \underbrace{\beta \bar{m}}_{\text{market exposure}} \quad (42)$$

Yearly rather than daily returns are used for a specific reason. The agents hold positions for days to weeks and act once per day, so a daily regression would measure how the model tracks the market within a holding period rather than whether it chose the right direction over the horizon it trades. Eleven yearly observations is a small sample and the individual coefficients carry wide uncertainty; what the decomposition supports is the separation of pairs into groups, not a precise value for any one of them.

Table 24 gives both terms. Six of the seven pairs have positive alpha, and the seven fall into three clear groups.

Three pairs carry almost no market exposure. GBP/USD, EUR/USD and USD/CHF have betas of -0.054 , $+0.266$ and -0.082 , and their market exposure contributes 0.06, 0.01 and 0.16 percentage points per year respectively. For GBP/USD, 11.64 of its 11.70 points per year are policy. These are the results that would survive a change in market direction, because there is almost no direction in them to change.

Pair	Beta	Alpha (%/yr)	Beta Return (%/yr)	Mean (%/yr)	Reading
AUD/USD	-2.013	+4.96	+3.22	+8.18	Policy plus leveraged trend
EUR/USD	+0.266	+6.62	+0.01	+6.63	Policy, no market exposure
GBP/USD	-0.054	+11.64	+0.06	+11.70	Policy, no market exposure
NZD/USD	-0.708	+2.79	+1.83	+4.62	Policy plus leveraged trend
USD/CAD	+2.393	+10.02	+4.21	+14.23	Policy plus leveraged trend
USD/CHF	-0.082	+3.91	+0.16	+4.07	Policy, no market exposure
USD/JPY	+1.012	-1.08	+2.64	+1.57	No policy; the model is the market

Table 24: Decomposition of Equation 42, from the yearly regression of Equation 41 over 2015 to 2025. Alpha is the return attributable to the policy and the beta return is the part explained by the pair's own movement. The two sum to the mean yearly return.

Three pairs combine policy with leveraged exposure. USD/CAD at +2.393, AUD/USD at -2.013 and NZD/USD at -0.708 all carry their market at meaningful size, two of them at more than twice. Their alpha is positive in every case, so there is skill on top, but between a fifth and two fifths of what they earned came from the market moving the way their exposure was pointed. USD/CAD is the clearest example: 4.21 of its 14.23 points per year came from the pair rising while the agent was net long. Had the Canadian Dollar strengthened over the decade instead, that component would have been a loss of similar size, and the reported +226.6% would look very different.

Figure 15 plots the seven together, and the three groups separate clearly.

One pair has no policy at all. USD/JPY has a beta of +1.012 and a correlation of +0.946 with its own market, and its alpha is -1.08% per year, the only negative figure in the table. It is examined in Section 5.9.

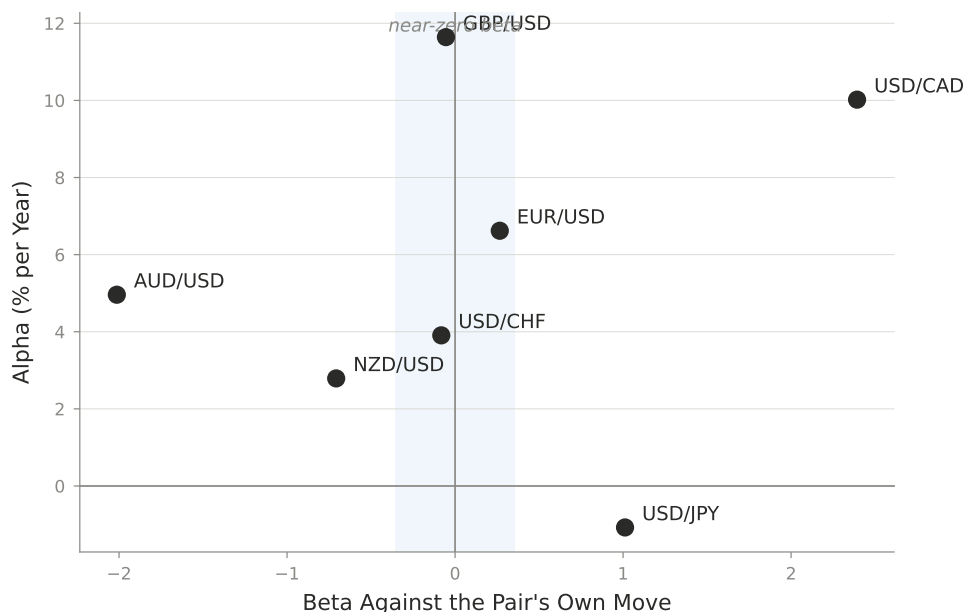


Figure 15: Alpha against beta for the seven agents. Points inside the shaded band carry almost no market exposure, so their return is attributable to the policy. USD/JPY sits at a beta of one with negative alpha, which is the signature of a model that has reproduced its own market.

The method was checked before it was applied. Running the same regression on the model delivered by the preceding campaign reproduces its recorded figures exactly, a correlation of +0.115, a beta of +0.130 and an alpha of +3.11% per year, which confirms that the decomposition here is computed the same way as the one it is compared against.

This decomposition is reported ahead of total return for a reason that matters to how the whole section should be read. The choice of which model to publish was made from a pool of candidates, and any measure that the choice was made on is inflated by that choice. Return is such a measure. Beta is not: it describes how a policy behaves against its market, and a model cannot be selected into a low beta by luck in the way it can be selected into a high return. A large alpha with a beta near zero is therefore a stronger claim than a large return, and it is the claim this work leads with.

5.7 Statistical Screening and the Rejected Candidates

Every published model had to pass a test against a null built from itself. The model’s own sequence of positions is rotated in time, so that it holds the same positions for the same durations but at the wrong moments, and the resulting regime score is the score its holding structure earns by luck alone. The published model must beat that score.

The null has to be computed per model rather than fixed. Measured across the seven pairs it ranges from 48.21 on GBP/USD to 69.82 on NZD/USD. A fixed threshold of, say, 55 would sit below chance on one pair and well above it on another, so it would accept noise in one market and reject skill in another. This was found by trying the fixed threshold first.

The screening produced a result that changes how the whole campaign should be read. **In every one of the seven pairs, the candidate with the highest return failed the test.** Table 25 lists the strongest cases. A portfolio built by ranking candidates on return would have selected models averaging roughly +250%, almost all of which is market exposure rather than policy.

Pair	Return	p -value	Why it was rejected
AUD/USD	+246.8%	0.146	Failed the screen
AUD/USD	+142.6%	0.987	Published during the campaign, then withdrawn when the test was run
EUR/USD	+141.6%	—	One-sided: long for 86% of the window
GBP/USD	+259.8%	0.163	Failed the screen
NZD/USD	+253.9%	0.258	Failed the screen
USD/CHF	+463.9%	0.973	Positions no better timed than the same positions shuffled; rode the franc trend
USD/CHF	+341.3%	0.865	As above

Table 25: Candidates rejected by the screening of Section 5.7. In every pair the highest-return candidate failed. Ranking on return alone would have selected these models over the ones published, at roughly five times the apparent return and almost none of the measurable skill.

The USD/CHF case is the clearest. A candidate returning +463.94%, more than nine times the published model, has a permutation p -value of 0.973: its positions are no better timed than the same positions shuffled. It rode the franc’s trend. The published model returns a ninth of that and is the one with measurable skill.

Two cases are worth reporting because they are uncomfortable. An AUD/USD candidate returning +142.55% was published during the campaign and then withdrawn when its p -value came out at 0.987. And on EUR/USD the incumbent model won on four of six metrics, including return, Sharpe, Sortino and drawdown, and was still replaced, because it lost on the one axis that decides whether a result is real.

The limits of this screening should be stated. Each published model was chosen from between 128 and 256 candidates, so a p -value near 0.05 is close to what that many draws produce by chance. Under a strict correction for the number of candidates examined, only USD/CAD and GBP/USD, both at 0.0015, would clearly survive. The permutation test is therefore used here as a screen that rejected bad models, which it demonstrably did, and not as a claim of statistical significance for the ones that remain.

5.8 Out-of-Sample Behaviour: The Held-Out Year

The split of Equation 32 trains on 2015 to 2023, selects among episodes on 2024, and leaves 2025 untouched. Because the trained weights are stored, 2025 can be scored on its own.

It is negative. Table 26 gives the figures: five of the seven pairs lose money in 2025, and the mean model return is -6.8% against a mean market move that would have made money. Only USD/CHF, at $+5.0\%$, and USD/JPY, at $+0.6\%$, are positive.

Pair	2025 (Test)	2024 (Validation)	Full Window	2025 Market
AUD/USD	-11.7%	$+72.4\%$	$+102.5\%$	$+7.6\%$
EUR/USD	-1.2%	-5.9%	$+73.9\%$	$+13.5\%$
GBP/USD	-22.9%	-3.6%	$+124.1\%$	$+7.6\%$
NZD/USD	-5.0%	$+11.0\%$	$+56.4\%$	$+2.8\%$
USD/CAD	-13.4%	$+28.8\%$	$+226.6\%$	-4.6%
USD/CHF	$+5.0\%$	$+3.7\%$	$+50.6\%$	-12.8%
USD/JPY	$+0.6\%$	$+7.6\%$	$+15.7\%$	-0.7%
Portfolio	-9.8%	—	$+91.3\%$	—

Table 26: The held-out year against the validation year and the full window. 2025 was seen by no part of the training or selection procedure; 2024 was used to choose which episode’s weights were kept. The last column is the move of the pair itself in 2025.

The contrast with 2024 is the important part, and Figure 16 shows it directly. 2024 is the best year in the whole sample, with a mean excess return over the benchmark of $+16.4$ percentage points. 2025 is the worst, at -8.7 . The difference between those two years is that 2024 was used to decide which episode’s weights were kept and 2025 was used for nothing. That is what selection pressure looks like when it is measured, and it is the reason the full-window figures in Table 19 are described as an in-sample result throughout this document.

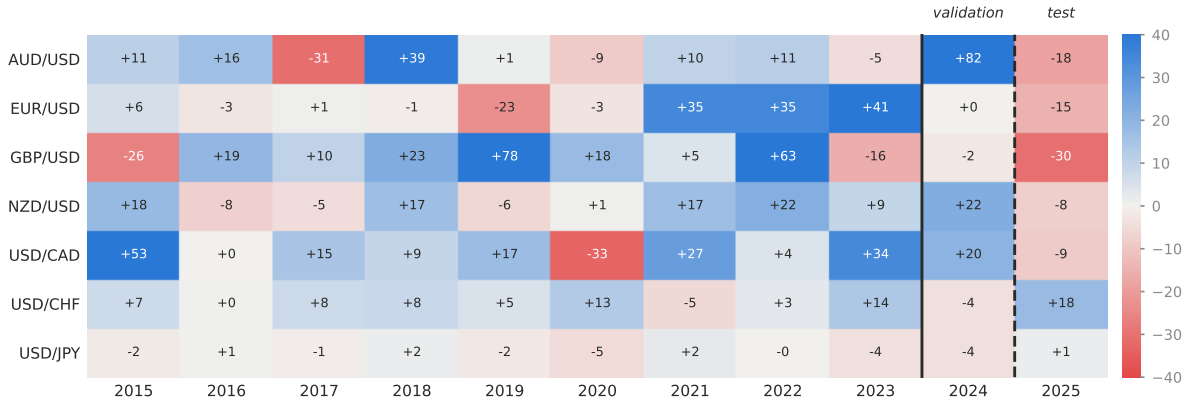


Figure 16: Yearly return of each agent minus the return of buying and holding the same pair, in percentage points. Blue is outperformance. The solid line marks the end of the training data and the dashed line the end of the validation year, so only the last column was unseen by any part of the procedure.

There is a market explanation for the direction of the 2025 result, and it is worth giving because it points somewhere. 2025 was a broad dollar reversal: EUR/USD rose 13.5% , its largest annual move in the eleven years, and the dollar weakened against five of the seven pairs. The agents were trained on a decade in which the dollar mostly strengthened, and six of the seven were positioned for that. The seventh, USD/CHF, is the least directional model in the set, holds the fewest positions and has the lowest beta, and it is the one that made money. That is consistent with the rest of the evidence rather than a coincidence: the model that depends least on a market direction is the one that survived the direction changing.

The individual results are worth reading rather than only their average, because they do not fail in the same way. GBP/USD is the worst at -22.9% , and it is the most aggressive model in the set, running near full size with the deepest drawdown over the full window; a year that goes against it costs

accordingly. USD/CAD loses 13.4% and AUD/USD 11.7%, and both are the leveraged-exposure models of Section 5.6, so a reversal in the direction their beta points is exactly the event that hurts them. EUR/USD loses only 1.2% despite the pair moving 13.5%, which is consistent with its near-zero beta: it did not participate in the reversal in either direction.

The two that held up are the two that were holding least. USD/CHF returned +5.0% while its pair fell 12.8%, an excess of 17.8 percentage points and its best year against the benchmark in the whole sample. USD/JPY returned +0.6%, which is not skill but the absence of exposure to the pairs that moved. Between them they describe the pattern: the models that depended least on a market direction were the ones that survived the direction changing.

One year is a small sample and no conclusion should be drawn from it beyond the obvious one. What the held-out year establishes is a bound, not a verdict: the design continued to trade sensibly on data it had never seen, and it lost money doing so in a year whose direction its training data did not contain.

5.9 A Negative Result: USD/JPY

USD/JPY did not work, and the reason is understood.

The numbers say the model is the market. Its correlation with the pair's yearly move is +0.946 and its beta is +1.012, so it carries the pair almost exactly one for one. Its alpha is -1.07% per year. It returns +15.7% over the eleven years against roughly +25% for simply buying and holding the pair, so it underperforms the benchmark that costs nothing to run. The model is long at every point in the window.

The mechanism is in the data. USD/JPY rose from roughly 120 to 150 over the period, with the move concentrated and largely one-directional. On such a series, holding a long position is close to optimal, and any deviation from it is punished. The reward signal therefore offers no gradient towards modulating exposure, and the agent settles on the simplest policy that captures the trend. The other six majors mean-revert enough over the same period that two-sided behaviour is rewarded, and their agents learned it.

This was tested rather than assumed. Across seven configurations and 448 seeds, every run converged to an all-in directional policy. Raising the augmentation that presents the agent with mirrored price series, which was intended to remove exactly this bias, made the resulting policies more binary rather than less.

The result is reported because it is informative. It marks the boundary of the method: this design learns to time exposure in markets that turn, and on a market that does not turn it reduces to holding the trend and pays costs for the privilege. A reader deciding whether to apply the approach elsewhere needs that boundary more than they need a seventh success.

5.10 Portfolio: The Seven Models Combined

A trader would not run one of these agents. They would run all seven, which is a different question from how each performs alone, because what matters for a portfolio is not only what each component returns but whether they lose money at the same time.

The seven published models are combined by giving each a seventh of the capital at the start of the window and never rebalancing. Writing $E_p(t)$ for the equity of pair p at time t , the portfolio is

$$V(t) = \frac{1}{7} \sum_p \frac{E_p(t)}{E_p(0)} \quad (43)$$

The total return of such a portfolio is the mean of its components by construction, so the return is not the interesting part. The risk is. Table 27 sets the portfolio against the average of the seven pairs taken separately.

Figure 17 plots the portfolio against its components. The maximum drawdown falls from an average of 34.5% to 15.9%, and the Sharpe ratio rises from an average of 0.451 to 0.632, above every individual pair except USD/CHF. The same return is delivered with less than half the worst loss.

Measure	Portfolio	Mean of the seven	Effect
Total return	+91.3%	+91.3%	Equal by construction
Annualised return	+6.07%	+6.07%	Equal by construction
Maximum drawdown	15.9%	34.5%	More than halved
Sharpe ratio	0.632	0.451	Above all but USD/CHF
Sortino ratio	0.906	0.685	Improved
Calmar ratio	0.381	0.194	Nearly doubled
Sterling ratio	1.059	0.670	Improved
Mean correlation	+0.062 across all 21 pairings		Near independence

Table 27: The equal-weight portfolio of Equation 43 against the average of its seven components. Return is identical by construction, so the whole effect is on risk.

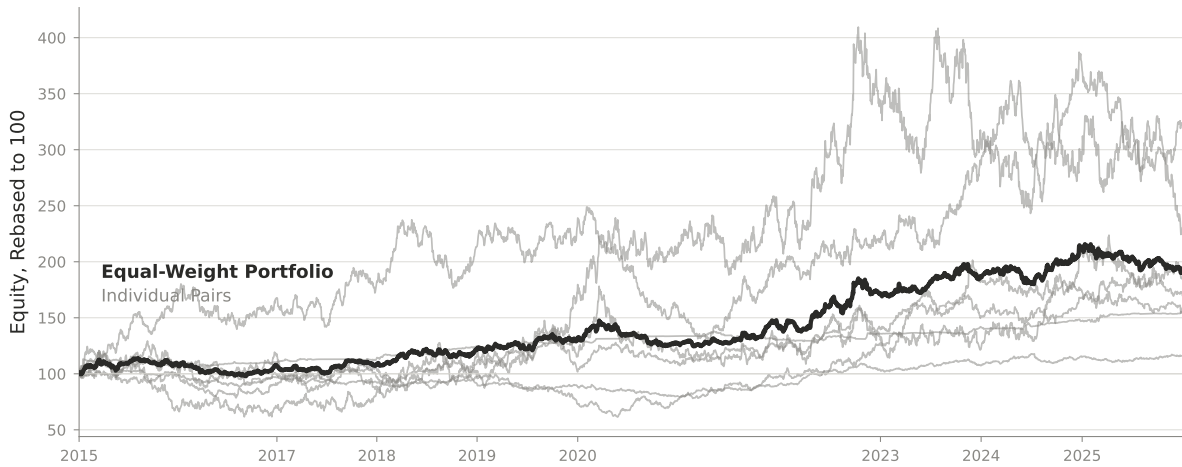


Figure 17: The equal-weight portfolio against its seven components, all rebased to 100. The portfolio is smoother than any component because the components do not fall together.

The reason is in Figure 18. The mean correlation between the daily returns of any two of the seven models is +0.062, and no pair of models exceeds +0.46. Three pairings are negative. These are seven agents trained by the same procedure on seven instruments that are themselves correlated, all sharing the US Dollar on one side, and yet their profits and losses arrive at different times. That is a consequence of the design: each agent learned the timing of its own market, so what they have in common is a method rather than a position.

5.10.1 Does the Result Depend on the Position Sizes?

The seven models do not all run at the same position size, as Section 5.1 states, so the portfolio above is the combination as published rather than one built on a common risk level. Whether that choice produces the result is a fair question and it can be answered rather than argued.

Position size is proportional to the account balance, so scaling the risk fraction by a constant scales every position and therefore every per-step return by the same constant. A common-risk variant can be constructed by rescaling each component's returns to the level it would have run at, and recomputing. This ignores the rounding of volumes to the broker's step and any margin limit, so it is an approximation rather than a re-run, but it is close enough to answer the question. Table 28 gives both versions.

The two portfolios are almost the same. Total return moves from +91.3% to +91.6%, the Sharpe ratio from 0.632 to 0.608 and the maximum drawdown from 15.9% to 19.0%. The common-risk version is slightly worse on risk, which is what should be expected: bringing USD/JPY onto the same footing as the others multiplies its positions fivefold and takes its own maximum drawdown to roughly 80%.

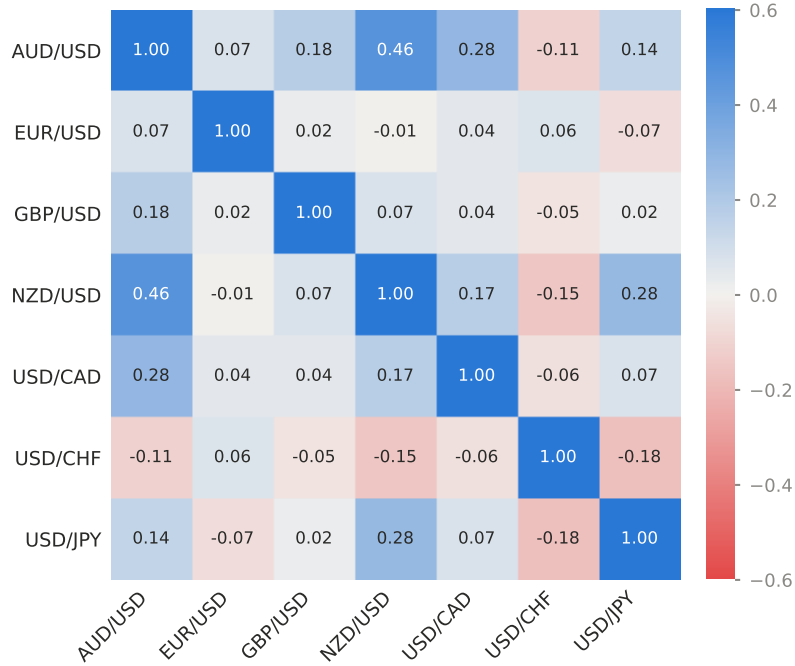


Figure 18: Correlation between the daily returns of the seven agents over the evaluation window. The mean of the off-diagonal entries is +0.062.

Measure	As Published	Common Risk	Difference
Total return	+91.3%	+91.6%	+0.3 pp
Annualised return	+6.07%	+6.09%	+0.02 pp
Sharpe ratio	0.632	0.608	-0.024
Sortino ratio	0.906	0.866	-0.040
Calmar ratio	0.381	0.320	-0.061
Maximum drawdown	15.9%	19.0%	+3.1 pp

Table 28: The equal-weight portfolio as published against a variant in which every component runs at a common risk level. The common-risk version is built by rescaling each component's per-step returns, which is exact for a strategy whose position size is proportional to balance but ignores volume rounding. The two differ little, so the diversification result does not depend on the sizing choice.

At a seventh of the capital, even that is absorbed. The conclusion of this subsection therefore does not depend on the sizing convention; it depends on the correlation structure, which sizing does not change.

Two qualifications belong with this result. The components do not all run at the same position size, as Section 5.1 states, so the portfolio is the combination as published rather than a risk-parity allocation; equalising the sizes would change the weights but not the correlation structure that produces the benefit. And the portfolio inherits the limitation of its components: in the held-out year of Section 5.8 it returns -9.8%, better than five of the seven pairs and worse than two, which is what an average should do when most of its components fall together for a reason they share.

Two results carry the most weight. Six of the seven agents produced positive alpha against their own market, and three of those did it with a market sensitivity close to zero, which is what separates a strategy from leveraged exposure. And the seven combined behave better than any of them alone, because they are close to uncorrelated.

Against that, the held-out year is negative and the selection is uncorrected. Section 6 sets out what those two facts allow the work to claim and what they do not.

6 Conclusion

This work set out to determine whether a single deep reinforcement learning design, trained separately for each instrument and otherwise unchanged, produces a profitable and risk-controlled policy on all seven major currency pairs when the evaluation charges what a real account charges.

The answer is a qualified yes, and the qualification is as much a part of the result as the answer.

6.1 Summary

Three things were built and one question was answered.

The first is a trading framework in which one strategy implementation runs in three places: an offline backtest over stored quotes, the cTrader backtesting tab, and a live account. Its execution engine reproduces the platform’s own results byte for byte across six reference configurations and extends them, stepping on ticks rather than bar closes and charging the spread recorded at the moment of each trade. Behind it sits a market database of 1.66 billion quotes across the seven pairs, occupying 295 GB, from which every bar series used in this work is derived.

The second is an agent. It observes thirty-eight dimensionless values built from sixteen technical indicators and emits one continuous action that sets the direction and the size of its position together. Six deviations from the published algorithm were needed to make it train on this problem, of which two matter: a scale-sensitive reward, because a scale-invariant one gives no gradient towards holding a meaningful position, and a penalty on the actor’s pre-activation, without which the policy saturates and collapses to a constant action.

The third is a selection procedure. Candidates are rejected by gates that remove degenerate policies, tested against a null built by rotating each model’s own position sequence in time, and ranked by a composite in which return is one component of nine. The procedure was not built to be elegant; it was built because ranking on return does not work, which Section 5.7 demonstrates rather than asserts.

The question was answered as follows. All seven agents are profitable over the eleven-year window and from all five starting balances, with total returns from +15.7% to +226.6% net of the spread and commission of a swap-free raw-spread account. Six of the seven produce positive alpha once their return is decomposed against their own market, and three of those carry almost no market exposure while doing it. The seventh, USD/JPY, reproduces its market and is reported as a failure with a diagnosed cause. Combining the seven into an equal-weight portfolio reduces the maximum drawdown from an average of 34.5% to 15.9% and raises the Sharpe ratio to 0.632, because the agents are close to uncorrelated with one another.

Two findings sit underneath those numbers and are more transferable than they are. The continuous action is genuinely used: the mean absolute action ranges from 0.09 to 0.98 across the pairs and saturation is below one percent everywhere, so the agents learned to size positions rather than to switch between extremes. And win rates lie between 49.8% and 55.0%, with the strongest model in the set having the lowest of them, so the edge does not come from being right more often. It comes from what the agents do with a position once it is open, which is the capability a continuous action provides and a discrete one does not.

6.2 Limitations

The most important limitation is stated first because it bounds everything else. The performance figures reported in Section 5 are measured over a window that includes the years the agents trained on. The procedure does hold a year back from training and from the choice among an agent’s own episodes, but the choice of which trained agent to publish was made on the full window, so the headline returns are an in-sample result. On the one year that no part of the procedure saw, the models lost money: the portfolio returned −9.8% in 2025 and five of the seven pairs were negative. That year contained a broad dollar reversal the training decade did not, and the one model that held up is the least directional of the set, but a single year is a small sample and nothing stronger should be read into it in either direction.

Selection is uncorrected in a second sense. Each published model was chosen from between 128 and 256 candidates, so a permutation p -value near 0.05 is close to what that many draws produce by chance. The screening is therefore reported as a filter that demonstrably rejected bad models, not as a significance claim for the ones that survived. Under a strict correction only USD/CAD and GBP/USD would clearly stand.

Three narrower limitations belong on the record. Position size was chosen per pair after results were seen; it scales every trade without changing the policy that generates them, but it is a parameter fitted on the same data as everything else. Training is not reproducible across environments: it is bit-exact within one installation at a single thread, and a change of numerical library version alters the outcome, so the delivered models are archived artifacts with a verified replay rather than a recipe that regenerates them. And two weeks of January 2016 are missing from all seven pairs for reasons on the broker’s side; the gap is uniform across pairs and falls outside the evaluation window, so it does not bias any comparison made here.

Finally, the scope is narrower than the method. Seven instruments in one asset class over one eleven-year period, with one decision cadence and one cost model, is a single point in a large space. Nothing in this work establishes that the design transfers to other markets, other periods or other holding horizons, and the USD/JPY result shows that it does not transfer to every instrument even within the seven.

6.3 Future Work

The limitations above order the work that follows, because the most useful next step is the one that would most change what the results are worth.

The first is a genuinely held-out evaluation. Training would end before the reporting period begins, and the choice among candidates would be made using only data from before that boundary, so that the reported figures are an out-of-sample estimate rather than a description of the window the models were selected on. This is the single change that would most strengthen the claims, and the framework already supports it; what it needs is a campaign run under the stricter protocol rather than new machinery.

The second follows from it. The procedure used here divides the window once. A walk-forward scheme with several folds, in which each fold trains on the data before it and is evaluated on the data after, would produce an out-of-sample record for the whole period rather than for a single year. Carrying the learned weights from one fold into the next, rather than starting each fold from a fresh initialisation, would test whether an agent adapts as the market changes; this is supported by the framework and was not exercised in this campaign.

The third is to report the variation between training runs. The path-robustness protocol used here varies the starting balance, which measures sensitivity to the entry path but not to the training itself. Reporting the distribution of outcomes across the candidate pool, and where the published model sits within it, would let a reader see how much of a result is method and how much is draw.

Beyond the evaluation, three directions follow from what the agents did. The portfolio of Section 5.10 weights the seven equally; weighting by risk, or by the correlation structure, should improve on that, and the near-independence of the components suggests the gain would be real. The observation is a single frame of thirty-eight values, so a recurrent or attention-based encoder that reads a window of history directly is the natural extension, and it was in the original plan for this work. And the framework already runs the same strategy on a live account, so a sustained live deployment would test the one thing no backtest can, which is whether the execution assumptions hold when the orders are real.

One improvement to the framework itself is worth recording. The execution engine charges the spread correctly, by filling at the ask and the bid, but it does not carry that charge into the exported trade record as a separate field, so recovering it requires the indirect calculation of Equation 40. Writing the spread cost alongside commission and swap on every trade would make the cost decomposition of Table 22 a direct read rather than an inference, and would allow it to be reported per position rather than only in aggregate.

The USD/JPY result deserves its own line. The failure there is understood as a consequence of a monotonic price window offering no gradient towards modulating exposure. Whether that is the whole

explanation is not established, and the pair should be revisited once the evaluation protocol above is in place.

Deep reinforcement learning applied to currency trading is neither the solved problem that some reported returns suggest nor the dead end that its out-of-sample record would imply. What this work shows is that the difficulty is not mainly in the learning algorithm. It is in the environment the agent is trained against, the costs it is charged, and above all in the discipline of deciding which of many trained models deserves to be believed. Six of seven agents produced returns that survive a decomposition into policy and market exposure; in every one of the seven the candidate that looked best on return did not.

Bibliography

- [1] BIS, “OTC foreign exchange turnover in April 2025,” Sep. 2025. [Online]. Available: <https://www.bis.org/publications/202509-commentary-otc-derivatives>
- [2] E. F. Fama and B. G. Malkiel, “EFFICIENT CAPITAL MARKETS: A REVIEW OF THEORY AND EMPIRICAL WORK*,” *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, May 1970. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1111/j.1540-6261.1970.tb00518.x>
- [3] T. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” *CoRR*, Sep. 2015. [Online]. Available: <https://www.semanticscholar.org/paper/Continuous-control-with-deep-reinforcement-learning-Lillicrap-Hunt/024006d4c2a89f7acacc6e4438d156525b60a98f>
- [4] E. P. Chan, *Quantitative Trading: How to Build Your Own Algorithmic Trading Business*. John Wiley & Sons, Jun. 2021, google-Books-ID: pWE0EAAAQBAJ.
- [5] P. Brandimarte, *An Introduction to Financial Markets: A Quantitative Approach*. John Wiley & Sons, Feb. 2018, google-Books-ID: _2tODwAAQBAJ.
- [6] B. Donnelly, *The Art of Currency Trading: A Professional’s Guide to the Foreign Exchange Market*. John Wiley & Sons, Jun. 2019, google-Books-ID: r8ybDwAAQBAJ.
- [7] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, Nov. 2018, google-Books-ID: sWV0DwAAQBAJ.
- [8] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. [Online]. Available: <https://www.nature.com/articles/nature14236>
- [9] T. G. Fischer, “Reinforcement learning in financial markets - a survey,” *Friedrich-Alexander-Universität Erlangen-Nürnberg, Institute for Economics*, 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Reinforcement-learning-in-financial-markets-a-Fischer/922864ede84bc49be4ac676951278a9b568b6383>
- [10] B. Hambly, R. Xu, and H. Yang, “Recent advances in reinforcement learning in finance,” *Mathematical Finance*, vol. 33, no. 3, pp. 437–503, Jul. 2023. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1111/mafi.12382>
- [11] T.-V. Pricope, “Deep Reinforcement Learning in Quantitative Algorithmic Trading: A Review,” *ArXiv*, May 2021. [Online]. Available: <https://www.semanticscholar.org/paper/Deep-Reinforcement-Learning-in-Quantitative-A-Pricope/a6a7e7be402fc5f7703e5c74a7b2afa84613a429>
- [12] M. M. López de Prado, *Advances in financial machine learning*. Hoboken, New Jersey: Wiley, 2018.
- [13] A. Alameer, H. Saleh, and K. Alshehri, “Reinforcement Learning in Quantitative Trading: A Survey,” *TechRxiv*, Mar. 2022. [Online]. Available: <https://www.techrxiv.org/doi/full/10.36227/techrxiv.19303853.v1>
- [14] J. Carapuço, R. Neves, and N. Horta, “Reinforcement learning applied to Forex trading,” *Applied Soft Computing*, vol. 73, pp. 783–794, Dec. 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1568494618305349>

- [15] C.-Y. Huang, “Financial Trading as a Game: A Deep Reinforcement Learning Approach,” *ArXiv*, Jul. 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Financial-Trading-as-a-Game%3A-A-Deep-Reinforcement-Huang/a13a070940d48bce111f31ba9c0233884a17ae62>
- [16] F. Rundo, “Deep LSTM with Reinforcement Learning Layer for Financial Trend Prediction in FX High Frequency Trading Systems,” *Applied Sciences*, vol. 9, no. 20, p. 4460, Oct. 2019. [Online]. Available: <https://www.mdpi.com/2076-3417/9/20/4460>
- [17] D. H. Bailey, J. M. Borwein, M. López De Prado, and Q. J. Zhu, “Pseudo-Mathematics and Financial Charlatanism: The Effects of Backtest Overfitting on Out-of-Sample Performance,” *Notices of the American Mathematical Society*, vol. 61, no. 5, p. 458, May 2014. [Online]. Available: <https://www.ams.org/jourcgi/jour-getitem?pii=noti1105>
- [18] “Forex & Algorithmic Trading Community | cTrader Community.” [Online]. Available: <https://ctrader.com/>
- [19] W. F. Sharpe, “The Sharpe Ratio,” *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, Oct. 1994. [Online]. Available: <http://pm-research.com/lookup/doi/10.3905/jpm.1994.409501>
- [20] F. A. Sortino and L. N. Price, “Performance Measurement in a Downside Risk Framework,” *The Journal of Investing*, vol. 3, no. 3, pp. 59–64, Aug. 1994. [Online]. Available: <http://pm-research.com/lookup/doi/10.3905/joi.3.3.59>
- [21] C. R. Bacon, *Practical portfolio performance measurement and attribution*, third edition ed., ser. Wiley finance. Hoboken, NJ: Wiley, 2023.
- [22] R. Rhoads, *Candlestick Charting For Dummies*. John Wiley & Sons, Jun. 2022, google-Books-ID: 9216EAAAQBAJ.
- [23] R. Teo, “Candlestick Pattern OHLC.” [Online]. Available: <https://www.tradingwithrayner.com/wp-content/uploads/2019/04/candlestick-pattern-ohlc.png>
- [24] M. A. Lim, *The Handbook of Technical Analysis + Test Bank: The Practitioner’s Comprehensive Guide to Technical Analysis*. John Wiley & Sons, Dec. 2015, google-Books-ID: N01_BwAAQBAJ.
- [25] S. Jansen, *Machine Learning for Algorithmic Trading*. Packt Publishing Ltd, Jul. 2020, google-Books-ID: 4f30DwAAQBAJ.
- [26] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997, google-Books-ID: EoYBngEACAAJ.
- [27] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, Nov. 2016, google-Books-ID: omivDQAAQBAJ.
- [28] R. Pramoditha, “Human Neuron vs Artificial Neuron.” [Online]. Available: https://miro.medium.com/v2/resize:fit:2902/format:webp/1*hkYITODpjJgo32DoCOWN5w.png
- [29] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. [Online]. Available: <https://doi.apa.org/doi/10.1037/h0042519>
- [30] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986. [Online]. Available: <https://www.nature.com/articles/323533a0>
- [31] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, vol. 2, no. 5, pp. 359–366, Jan. 1989. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/0893608089900208>

- [32] B. F. Skinner, "Operant behavior," *American Psychologist*, vol. 18, no. 8, pp. 503–515, 1963, place: US Publisher: American Psychological Association.
- [33] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, "Playing Atari with Deep Reinforcement Learning," *ArXiv*, Dec. 2013. [Online]. Available: <https://www.semanticscholar.org/paper/Playing-Atari-with-Deep-Reinforcement-Learning-Mnih-Kavukcuoglu/2319a491378867c7049b3da055c5df60e1671158>
- [34] M. Riedmiller, "Neural Fitted Q Iteration – First Experiences with a Data Efficient Neural Reinforcement Learning Method," *Machine Learning: ECML 2005*, vol. 3720, pp. 317–328, 2005, series Title: Lecture Notes in Computer Science. [Online]. Available: http://link.springer.com/10.1007/11564096_32
- [35] H. Van Hasselt, A. Guez, and D. Silver, "Deep Reinforcement Learning with Double Q-Learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, Mar. 2016, iSSN: 2374-3468, 2159-5399 Issue: 1 Journal Abbreviation: AAAI. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/10295>
- [36] T. L. Meng and M. Khushi, "Reinforcement Learning in Financial Markets," *Data*, vol. 4, no. 3, p. 110, Jul. 2019. [Online]. Available: <https://www.mdpi.com/2306-5729/4/3/110>
- [37] A. R. Azhikodan, A. G. K. Bhat, and M. V. Jadhav, "Stock Trading Bot Using Deep Reinforcement Learning," *Springer*, vol. 32, pp. 41–49, 2019, book Title: Innovations in Computer Science and Engineering Series Title: Lecture Notes in Networks and Systems. [Online]. Available: http://link.springer.com/10.1007/978-981-10-8201-6_5
- [38] P. K. Gran, A. J. K. Holm, and S. Søgård, "A Deep Reinforcement Learning Approach to Stock Trading," *NTNU*, 2019. [Online]. Available: <https://www.semanticscholar.org/paper/A-Deep-Reinforcement-Learning-Approach-to-Stock-Gran-Holm/ef9f709e845d282cfb9371708d05d61580d1e10c>
- [39] M. Kong and J. So, "Empirical Analysis of Automated Stock Trading Using Deep Reinforcement Learning," *Applied Sciences*, vol. 13, no. 1, p. 633, Jan. 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/1/633>
- [40] Y. Li, W. Zheng, and Z. Zheng, "Deep Robust Reinforcement Learning for Practical Algorithmic Trading," *IEEE Access*, vol. 7, pp. 108 014–108 022, 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8786132/>
- [41] N. Majidi, M. Shamsi, and F. Marvasti, "Algorithmic Trading Using Continuous Action Space Deep Reinforcement Learning," *Expert Systems with Applications*, 2022, publisher: arXiv Version Number: 1. [Online]. Available: <https://arxiv.org/abs/2210.03469>
- [42] H. K. Sagiraju and S. Mogalla, "Application of multilayer perceptron to deep reinforcement learning for stock market trading and analysis," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 24, no. 3, p. 1759, Dec. 2021. [Online]. Available: <http://ijeecs.iaescore.com/index.php/IJECS/article/view/24816>
- [43] J. Sim and B. Kirk, "Generalized Deep Reinforcement Learning for Trading," *Journal of Student Research*, vol. 12, no. 1, Feb. 2023. [Online]. Available: <https://www.jsr.org/hs/index.php/path/article/view/4316>
- [44] T. Théate and D. Ernst, "An application of deep reinforcement learning to algorithmic trading," *Expert Systems with Applications*, vol. 173, p. 114632, Jul. 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417421000737>

- [45] X. Wu, H. Chen, J. Wang, L. Troiano, V. Loia, and H. Fujita, "Adaptive stock trading strategies with deep reinforcement learning methods," *Information Sciences*, vol. 538, pp. 142–158, Oct. 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025520304692>
- [46] Z. Xiong, X.-Y. Liu, S. Zhong, H. Yang, and A. Elwalid, "Practical Deep Reinforcement Learning Approach for Stock Trading," *ArXiv*, Nov. 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Practical-Deep-Reinforcement-Learning-Approach-for-Xiong-Liu/412fbb7ffa29aa6b2e172c1f545223baf18f1b6e>
- [47] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, "Deep reinforcement learning for automated stock trading: an ensemble strategy," *Proceedings of the First ACM International Conference on AI in Finance*, pp. 1–8, Oct. 2020, conference Name: ICAIF '20: ACM International Conference on AI in Finance ISBN: 9781450375849 Place: New York New York Publisher: ACM. [Online]. Available: <https://dl.acm.org/doi/10.1145/3383455.3422540>
- [48] N. Yousefi, "Deep Reinforcement Learning for Tehran Stock Trading," *Journal of Novel Engineering Science and Technology*, vol. 1, no. 02, pp. 37–42, Nov. 2022. [Online]. Available: <https://journal.iistr.org/index.php/JNEST/article/view/171>
- [49] L. Chen and Q. Gao, "Application of Deep Reinforcement Learning on Automated Stock Trading," *2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS)*, pp. 29–33, Oct. 2019, conference Name: 2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS) ISBN: 9781728109459 Place: Beijing, China Publisher: IEEE. [Online]. Available: <https://ieeexplore.ieee.org/document/9040728/>
- [50] Z. Jia, Q. Gao, and X. Peng, "LSTM-DDPG for Trading with Variable Positions," *Sensors*, vol. 21, no. 19, p. 6571, Sep. 2021. [Online]. Available: <https://www.mdpi.com/1424-8220/21/19/6571>
- [51] Z. Zhang, S. Zohren, and S. Roberts, "Deep Reinforcement Learning for Trading," *The Journal of Financial Data Science*, vol. 2, pp. 25–40, Apr. 2020, iSSN: 2640-3943 Issue: 2 Journal Abbreviation: JFDS. [Online]. Available: <http://pm-research.com/lookup/doi/10.3905/jfds.2020.1.030>